

Indigenous Environmental Data

Science

Teacher Guide



About Us

These slides are a collaboration between:

[Victoria Thomson](#), an Albert Einstein Distinguished Educator Fellow with USGS and high school science teacher, Christine Martin, PhD Candidate at Montana State University, and Rachel Torres, a postdoctoral scholar at Cal Poly Humboldt

The larger project and coding activities were created through a collaboration between Georgia Smies, an instructor at Salish Kootenai College, Sean Dorr, Jason McLachlan, and students at the University of Notre Dame. It is funded by the Alfred P. Sloan Foundation.

Our goal is to increase equitable pathways in earth and environmental data science.



Time and Structure of Indigenous Environmental Data Science



Piloted at Salish Kootenai College in Montana Winter Quarter 2024
Course: Intro to R Statistics in the Wildlife and Fisheries Dept.

Task: Complete Modules 1 - 3



Each Module requires ~3 weeks (6 hr class time) + outside class time

Frequency of Meetings:

Meet 1x per week for 2 hours

Total of 10 weeks



Class Format:

First hour of class: direct instruction/discussion



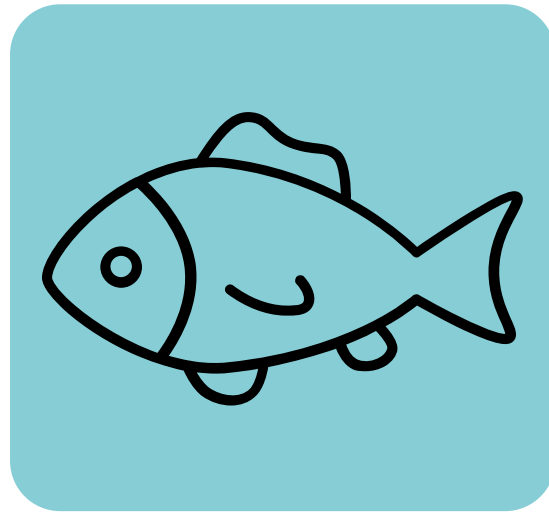
Background and Purpose



- Why are we doing this?
 - To introduce you to a fictional tribe K'avi so you can analyze data relevant to indigenous communities
 - Code can appear scary; but the sooner you play with it, you will see it's not so bad! It can be really powerful for you to know and understand how to use this programming language
 - To empower you to ask questions that are relevant to your community and identify how to share the information learned from your questions
- What is data science?
 - Data science is an interdisciplinary field that uses scientific methods, processes, algorithms, and systems to extract knowledge and insights from data (source: [datacamp](#))
- What is R?
 - R is a programming language commonly used to analyze large sets of data



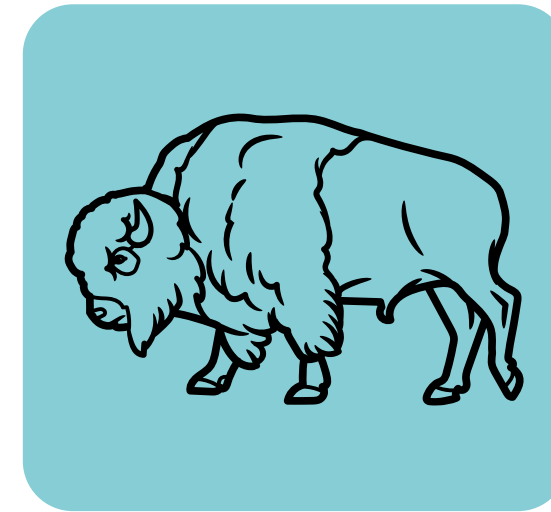
Indigenous Environmental Data Science Modules



Module 1: Mercury
in Fish



Module 2: Water
Quality



Module 3: Bison
Reintroduction



Module 4: Forestry
Management



[Github](#)



GitHub is like a big online notebook where people can save and share their coding projects.

Imagine you're working on a school project with friends. Instead of passing a notebook back and forth, GitHub lets you all work on it at the same time and saves every change.

It helps people:

- ✓ Store their code safely
- ✓ Work together on projects
- ✓ Keep track of all changes

Think of it as Google Docs, but for coding! 🚀



[Environmental Indigenous Data Science
Code on Github](#)

Choose Your Path



[R and R studio](#)
[Free, but requires](#)
[downloading software](#)

[Best for:](#)
[Laptops with memory](#)
[Computer labs](#)



[binder](#)
[Free, cloud-based service](#)
[to run notebooks](#)

[Best for:](#)
[Any environment](#)



Day

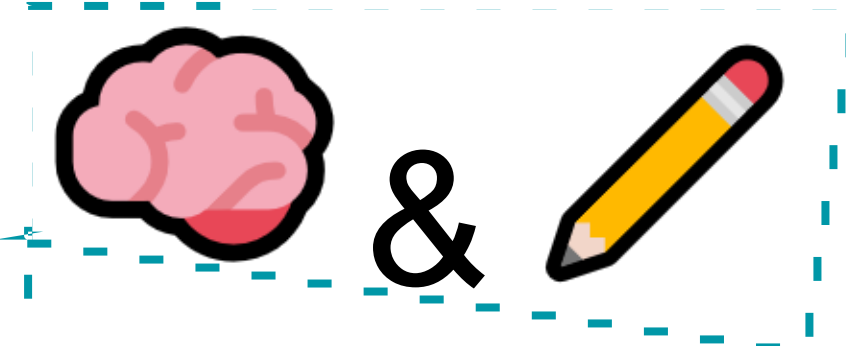
1

Lesson 1.1: R Basics & Calculating Screening Values



Goals:

- Identify the function name that loads packages
- Recognize objects including variables, vectors, and data frames
- Calculate values using mathematical operations and functions
- Generate a scatterplot using ggplot2
- Describe the relationships between body weight, weekly servings, and screening values (safe limits)



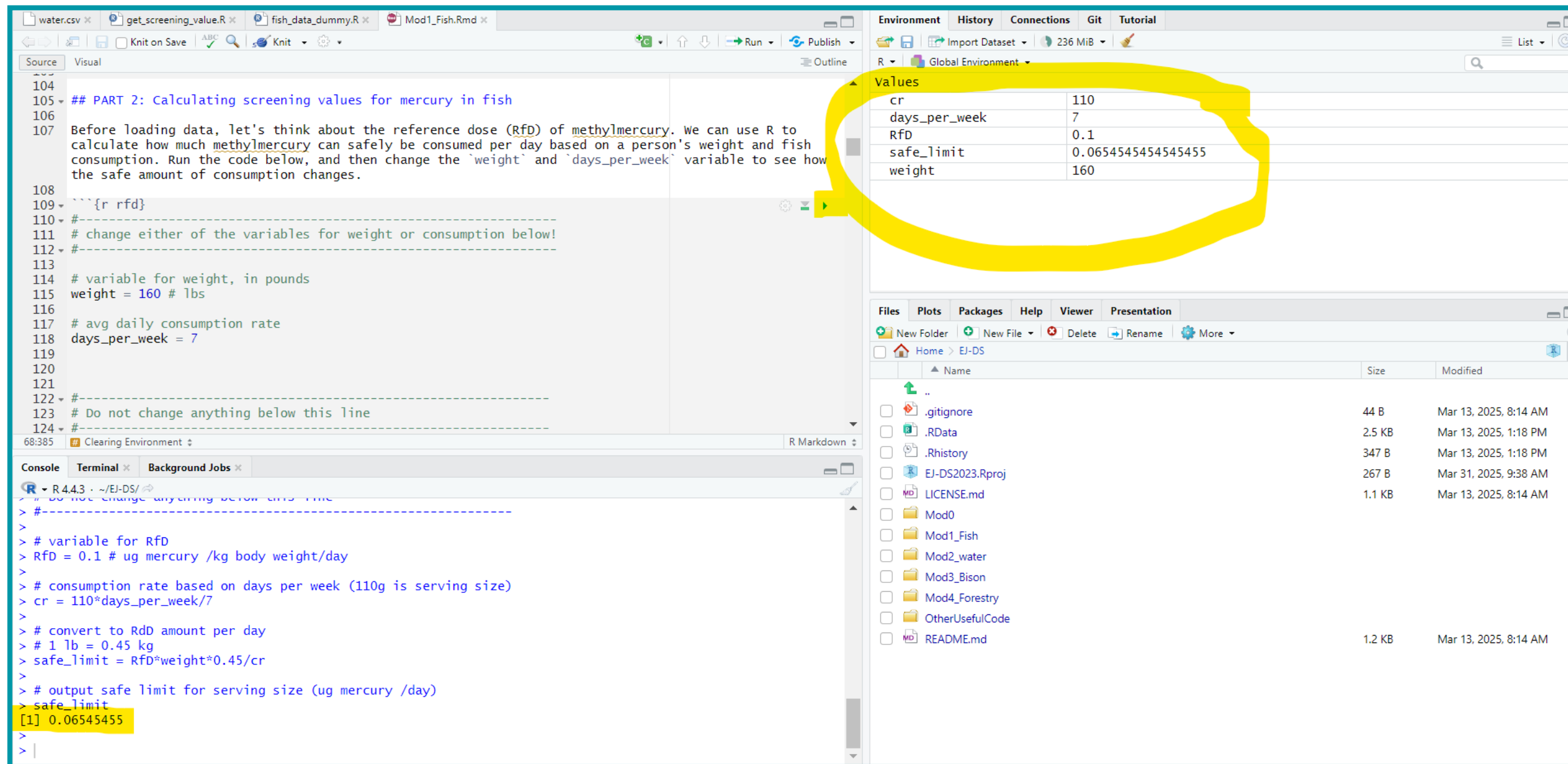
Loading packages

What is the name of the package that we will install and load for this project?

Calculate Screening Values for Mercury in Fish

Run the code (press play ) in the Part 2 section to determine the safe limit of consumption.

Your results will appear in both the console and the environment windows.



The screenshot displays the R Studio interface with the following components:

- Source Editor:** Contains R code for calculating screening values. The code includes comments and variable assignments for `weight`, `days_per_week`, `RfD`, `cr`, and `safe_limit`. A green play button is visible next to the code block.
- Environment Window:** Shows the current environment with variables `cr`, `days_per_week`, `RfD`, `safe_limit`, and `weight` and their respective values.
- Console Window:** Displays the output of the code execution, showing the calculation of `safe_limit` as `[1] 0.06545455`.
- Files Panel:** Shows the project structure, including files like `.gitignore`, `.RData`, `.Rhistory`, `EJ-DS2023.Rproj`, `LICENSE.md`, `Mod0`, `Mod1_Fish`, `Mod2_water`, `Mod3_Bison`, `Mod4_Forestry`, `OtherUsefulCode`, and `README.md`.

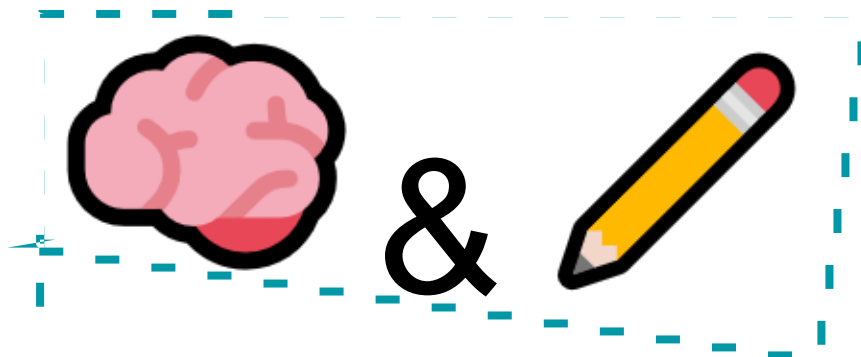
Variable	Value
<code>cr</code>	110
<code>days_per_week</code>	7
<code>RfD</code>	0.1
<code>safe_limit</code>	0.0654545454545455
<code>weight</code>	160

```
## PART 2: Calculating screening values for mercury in fish
Before loading data, let's think about the reference dose (RfD) of methylmercury. We can use R to
calculate how much methylmercury can safely be consumed per day based on a person's weight and fish
consumption. Run the code below, and then change the 'weight' and 'days_per_week' variable to see how
the safe amount of consumption changes.

## {r rfd}
#-----
# change either of the variables for weight or consumption below!
#-----
# variable for weight, in pounds
weight = 160 # lbs
# avg daily consumption rate
days_per_week = 7
#-----
# Do not change anything below this line
#-----

# variable for RfD
RfD = 0.1 # ug mercury /kg body weight/day
# consumption rate based on days per week (110g is serving size)
cr = 110*days_per_week/7
# convert to RfD amount per day
# 1 lb = 0.45 kg
safe_limit = RfD*weight*0.45/cr
# output safe limit for serving size (ug mercury /day)
safe_limit
[1] 0.06545455
```





Change the value for weight or days per week, and record the changes to the safe limit on your student worksheet.

```
water.csv x get_screening_value.R x fish_data_dummy.R x Mod1_Fish.Rmd x
Source Visual
104
105 ## PART 2: Calculating screening values for mercury in fish
106
107 Before loading data, let's think about the reference dose (RfD) of methylmercury. We can use R to
    calculate how much methylmercury can safely be consumed per day based on a person's weight and fish
    consumption. Run the code below, and then change the `weight` and `days_per_week` variable to see how
    the safe amount of consumption changes.
108
109 ```{r rfd}
110 #-----
111 # change either of the variables for weight or consumption below!
112 #-----
113
114 # variable for weight, in pounds
115 weight = 160 # lbs
116
117 # avg daily consumption rate
118 days_per_week = 7
119
120
121
122 #-----
123 # Do not change anything below this line
124 #-----
68:385 # Clearing Environment R Markdown
```



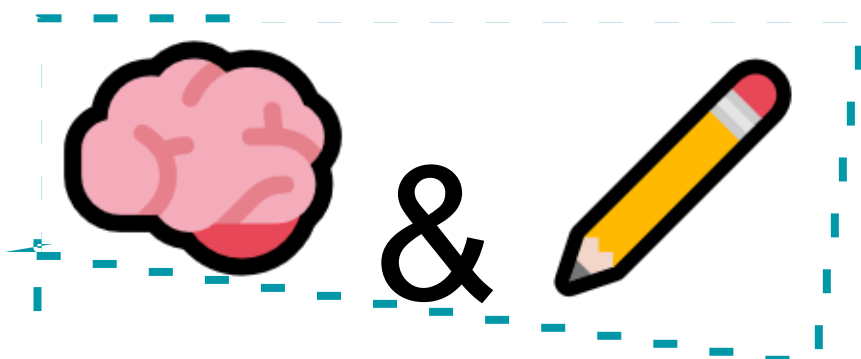
Creating Objects

An object is like a container or a box that holds data or information. You can give this box a name, and then use that name to look at or change what's inside the box later.

Example:

Imagine you want to store your age in R. You can create an object called `my_age` and put the number 25 in it.

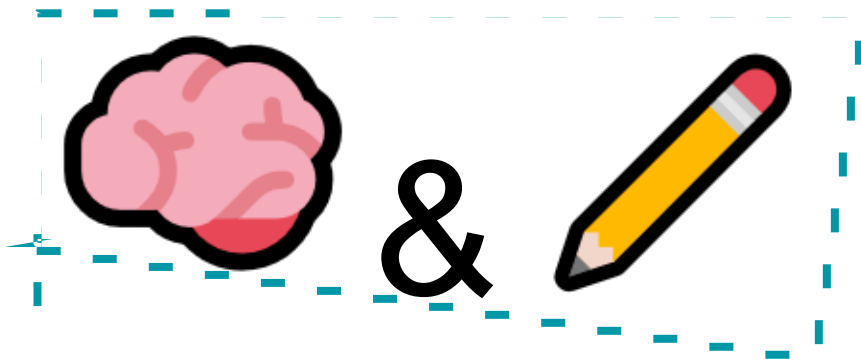
Which objects were
created to calculate
consumption rate?



Creating Vectors

A vector is a list of the same type of object (i.e., numbers, words, TRUE/FALSE).

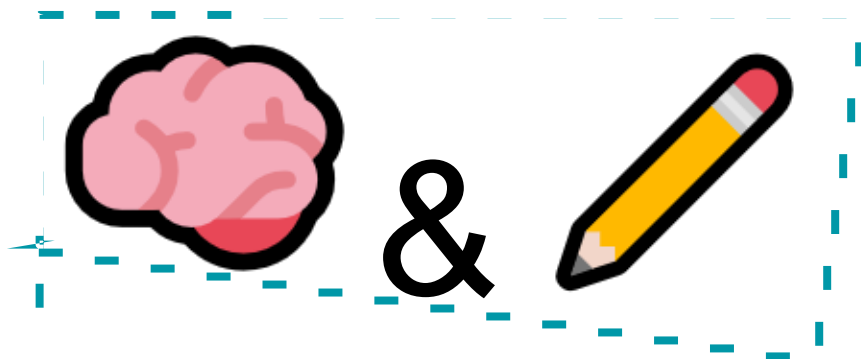
A vector is an object, but not all objects are vectors.



What function was used to
create the vectors?

Creating a Data Frame from Vectors

A data frame in R is a way to organize data in a table, similar to a spreadsheet. In a data frame, each column holds a group of related values (like weight, servings, and screening value), and each row represents one item or person.



Provide an example of a time when you have made a spreadsheet.

A data.frame: 3 × 3

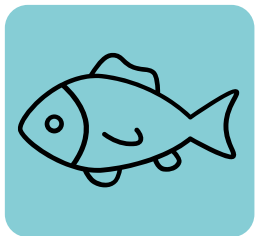
	weight	servings	screen_val
	<dbl>	<dbl>	<dbl>
1	140	1	0.40090909
2	175	6	0.08352273
3	200	3	0.19090909



Visualizing Screening Value vs. Servings Per Week with ggplot



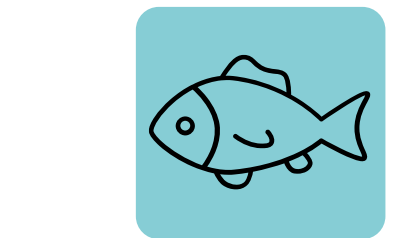
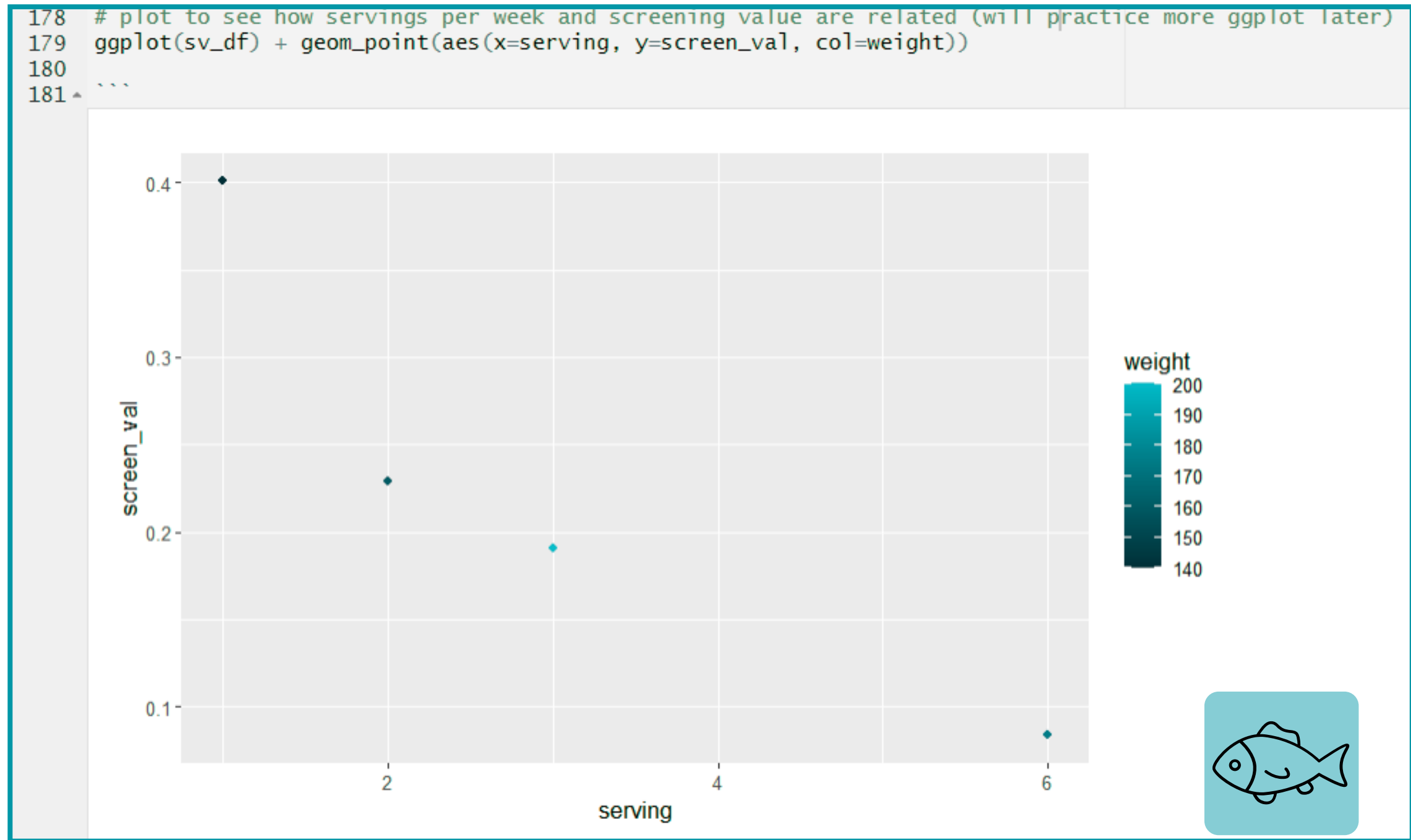
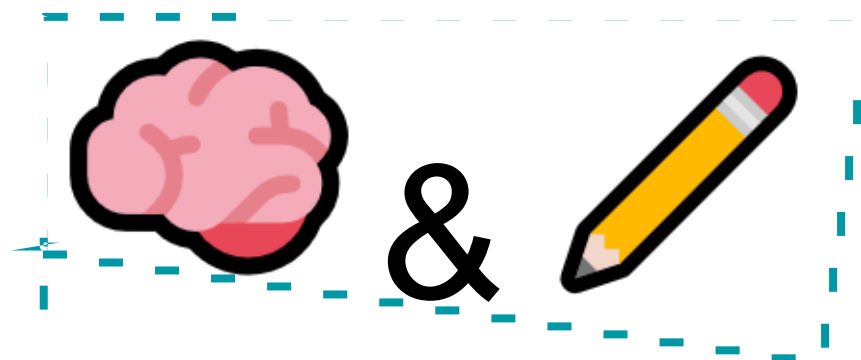
Visualizing Screening Value vs. Servings Per Week with ggplot



Identify and describe relationships in a graph

▶ Run the next section of code to create a data table (aka data frame) and graph. The following variables are included: weight, servings per week, and screening value.

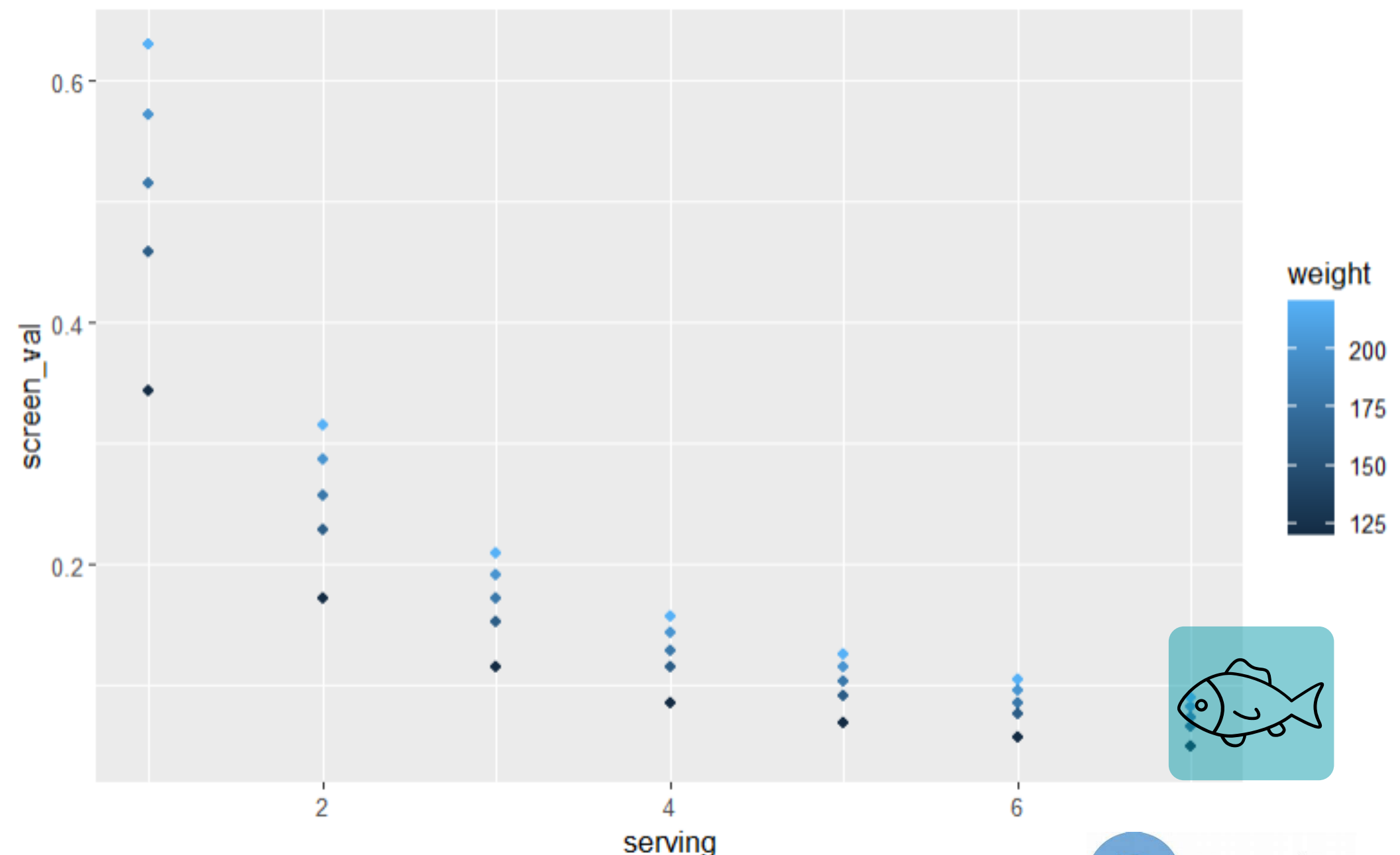
What is the relationship between screening value and the number of servings?

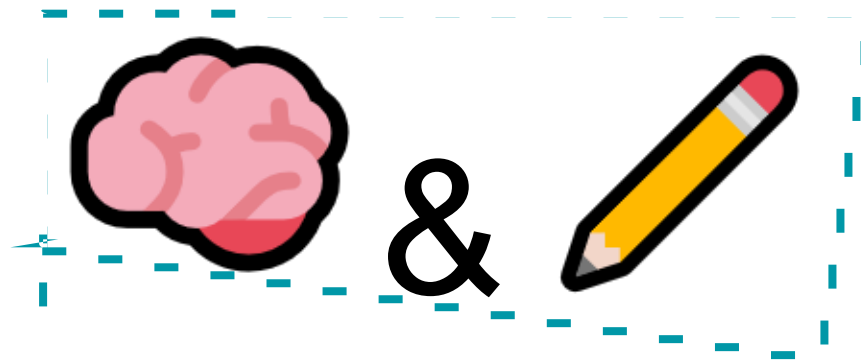


Identify and describe relationships in a graph

▶ Run the next section of code to create another graph; this time you will add have 5 weight categories.

```
187 sv_df_long = data.frame(weight, serving)  
188 sv_df_long$screen_val = RfD*sv_df_long$weight*0.45/(110*sv_df_long$serving/7)  
189 ggplot(sv_df_long) + geom_point(aes(x=serving, y=screen_val, col=weight))  
190  
191
```

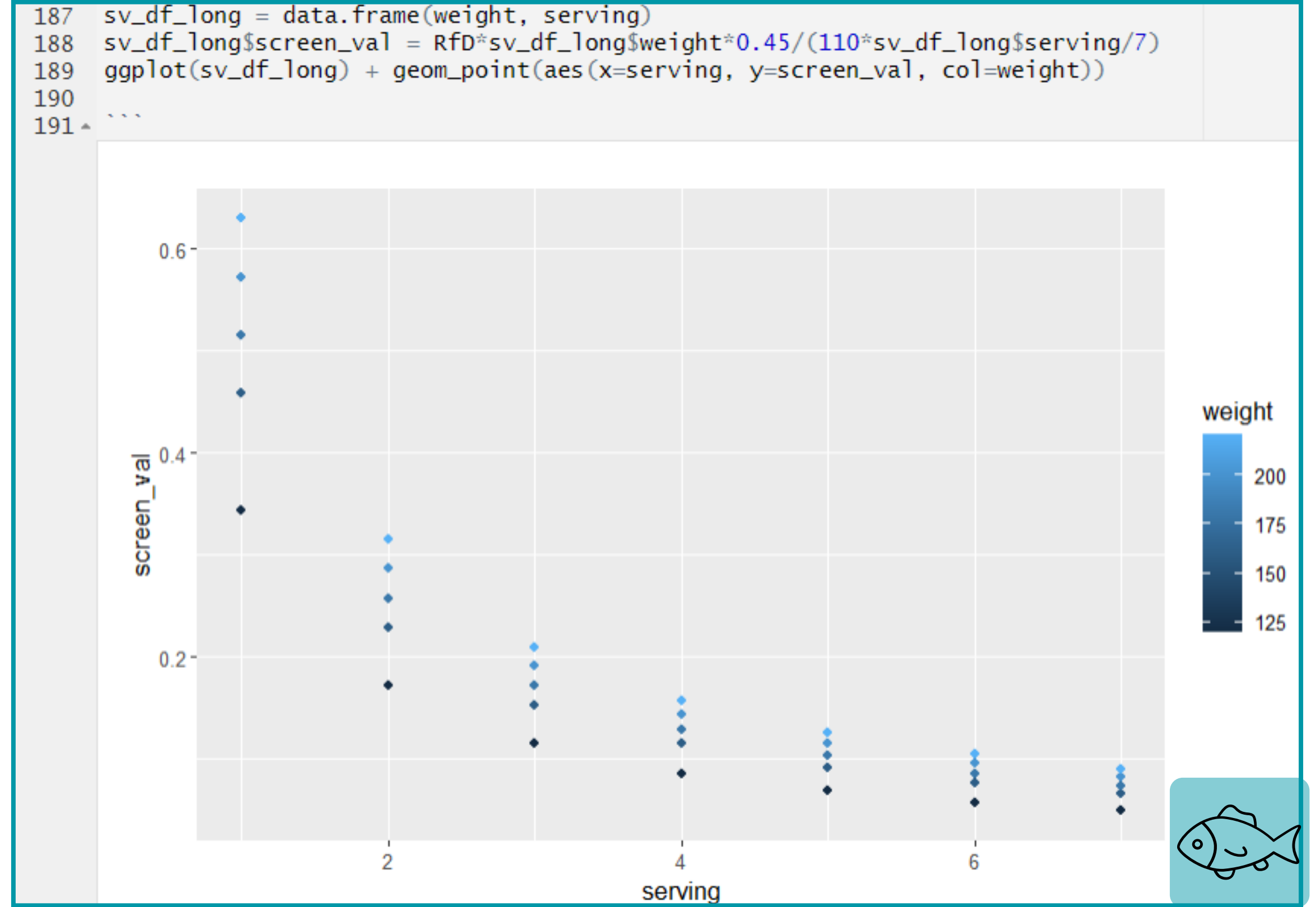




How would you describe this graph and the relationship between screening value, consumption rate, and weight?

What happens to the distribution in screening values as consumption rate gets larger?

What level should be considered dangerous? How do we know if there is a problem?



Lesson 1.2: Load in and Explore 1998 Fish Data

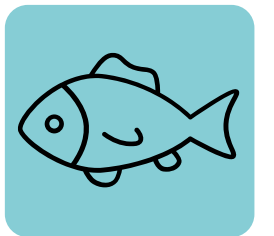
Goals:

- Load the 1998 fish data set
- View the entire data set in spreadsheet form
- View a truncated version of the data set
- Identify the number of rows and columns in the 1998 fish data set



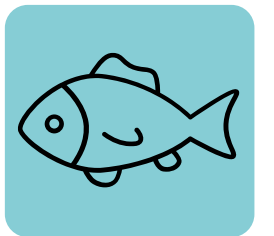
Run the code to load in fish data

```
216 **Now we will load in the dataset for the physical fish data**
217
218 These data are the length and methyl mercury content of fish caught on the K'Avi
    reservation in 1998, soon after the polluting factory was shut down.
219
220 * Each row is data from one fish.
221 * The first column "length" is the length of the fish in cm.
222 * The second column "mercury" is the concentration of methylmercury (a poison) in the
    fish, in units of micrograms (ug) of methylmercury per gram of fish (g).
223
224 ```{r fish_1998}
225 fish_1998 <- read.csv("fish_1998.csv", header = TRUE, sep = ',')
226 ```
227
```

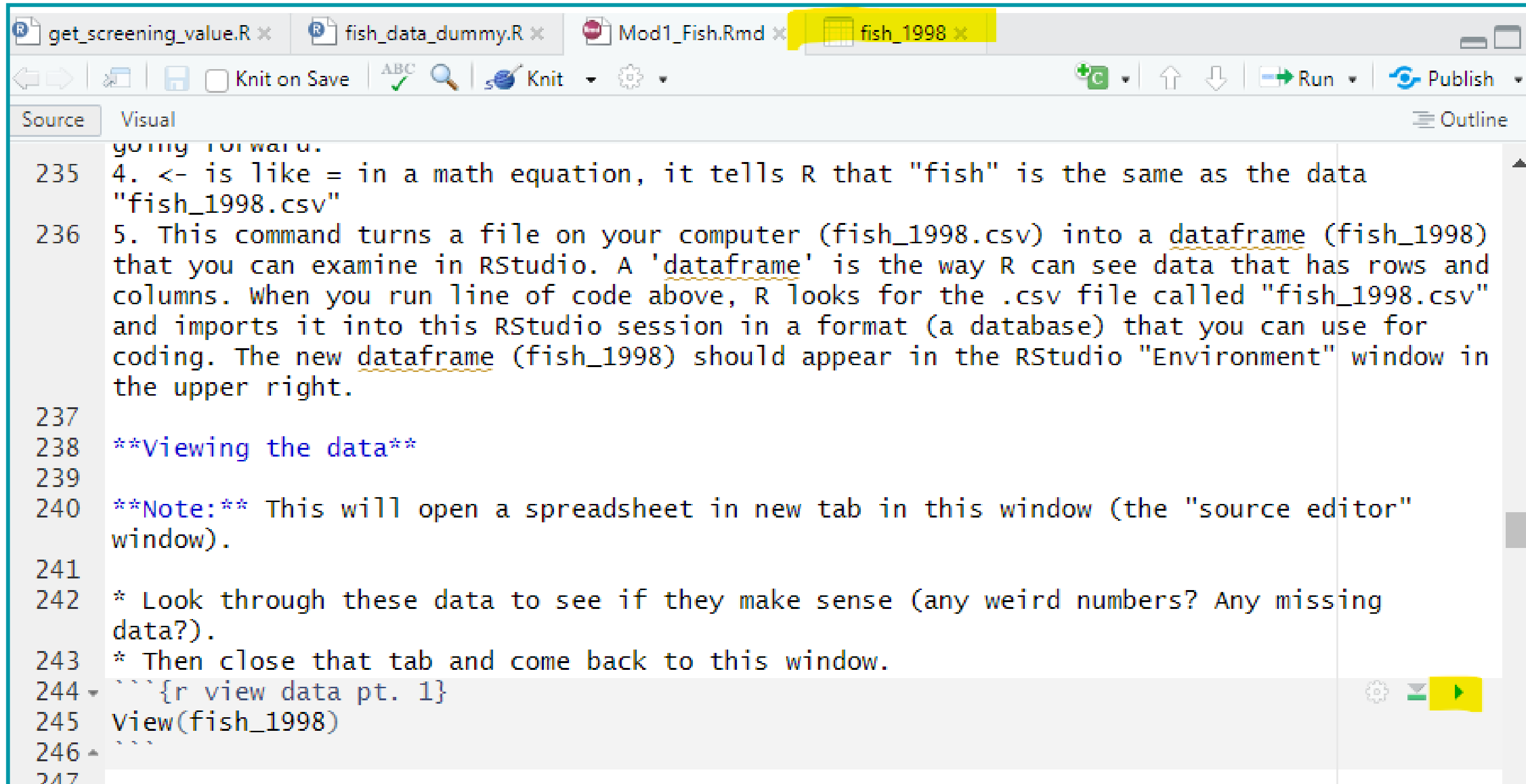


What did R do when we selected “run code”?

```
224 - ```{r fish_1998}
225 fish_1998 <- read.csv("fish_1998.csv", header = TRUE, sep = ',')
226 -
227
228 Let's break down the different parts of what just happened:
229
230 1. We told R to read the data (a .csv file) using the command read.csv()
231 2. Within the (""), we identify the data file from our working directory that we want to
   load into R
232 3. header = TRUE just tells R that the first row of data is the column names. The
   alternative is header = FALSE
233 4. sep is short for the word "separate", and it tells R that each column in the csv is
   separated by a comma ','
234 3. "fish" is the name we are giving that file in R, and is how we will refer to this data
   going forward.
235 4. <- is like = in a math equation, it tells R that "fish" is the same as the data
   "fish_1998.csv"
236 5. This command turns a file on your computer (fish_1998.csv) into a dataframe (fish_1998)
   that you can examine in RStudio. A 'dataframe' is the way R can see data that has rows and
   columns. When you run line of code above, R looks for the .csv file called "fish_1998.csv"
   and imports it into this RStudio session in a format (a database) that you can use for
   coding. The new dataframe (fish_1998) should appear in the RStudio "Environment" window in
   the upper right.
```



View the fish data in a spreadsheet form



The screenshot shows the RStudio source editor with the 'fish_1998' file selected. The editor displays R code comments and commands for loading and viewing data. The code is as follows:

```
235 4. <- is like = in a math equation, it tells R that "fish" is the same as the data
    "fish_1998.csv"
236 5. This command turns a file on your computer (fish_1998.csv) into a dataframe (fish_1998)
    that you can examine in RStudio. A 'dataframe' is the way R can see data that has rows and
    columns. When you run line of code above, R looks for the .csv file called "fish_1998.csv"
    and imports it into this RStudio session in a format (a database) that you can use for
    coding. The new dataframe (fish_1998) should appear in the RStudio "Environment" window in
    the upper right.
237
238 **Viewing the data**
239
240 **Note:** This will open a spreadsheet in new tab in this window (the "source editor"
    window).
241
242 * Look through these data to see if they make sense (any weird numbers? Any missing
    data?).
243 * Then close that tab and come back to this window.
244 ```{r view data pt. 1}
245 view(fish_1998)
246 ```
247
```



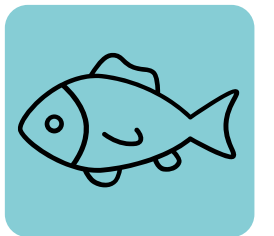
View a Truncated Set of the Fish Data

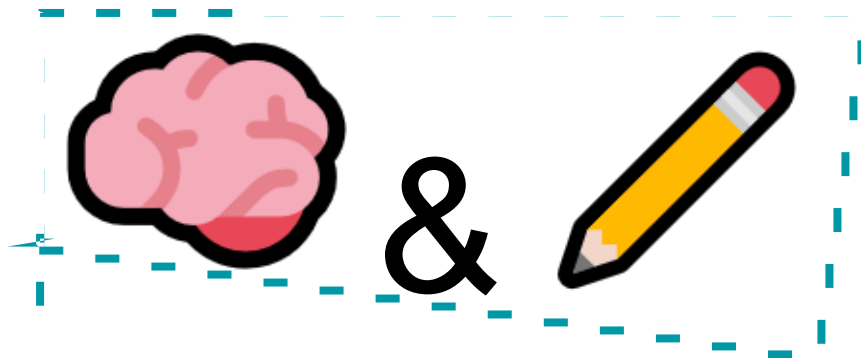
```
248 **A shorthand way to view the data**
249
250 If we don't want to look at all of the data and instead just want to see what is going on
    in the first couple rows we can use the following code.
251
252 The command head() shows the column names and the first few rows of the data
253
254 {r view data pt. 2}
255 head(fish_1998)
256
```

Description: df [6 × 3]

	X <int>	length <dbl>	mercury <dbl>
1	1	45.21643	0.4315457
2	2	44.34876	0.4969477
3	3	46.86563	0.4491865
4	4	41.67329	0.4137213
5	5	51.34514	0.7491135
6	6	56.66428	0.6807240

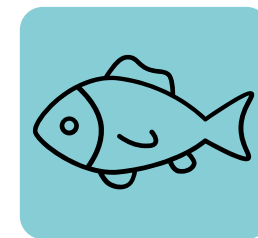
6 rows





What are the column titles in the R dataframe called fish_1998?

Identify at least 2 questions that the tribal fishery managers could answer with this data.



Identify the number of columns and rows in the data set

****Exploring the dimensions of the data****

To answer these questions, it would be helpful to know how many fish have methyl mercury measurements. We can very quickly find that out with the following code.

The command `dim()` shows the dimension of the dataset. "Dimension" is the number of rows & number of columns.

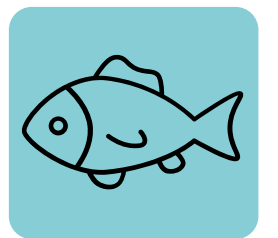
```
```{r explore data dimensions}
```

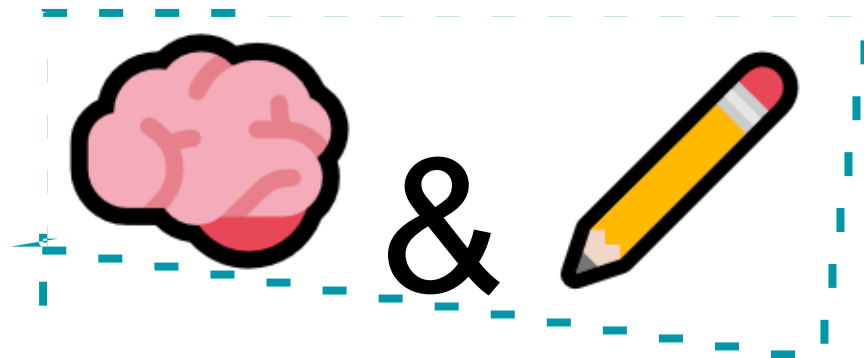
```
dim(fish_1998) #shows the dimensions of the dataset, which will be important later on
```

```
[1] 300 3
```

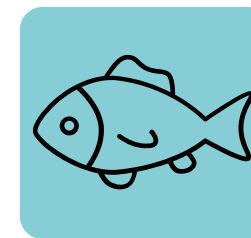
rows

columns





In 1998, how many fish did the fish managers measure for mercury?



# Lesson 1.3: Calculating Descriptive Statistics



Recall the maximum safe level of mercury to consume depends on how often fish are being consumed, and the weight of the person. According to the EPA, the maximum safe level of mercury to be consumed is around 0.46 ug/g/day if not frequently consuming fish.

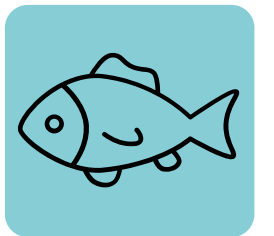
## Goal:

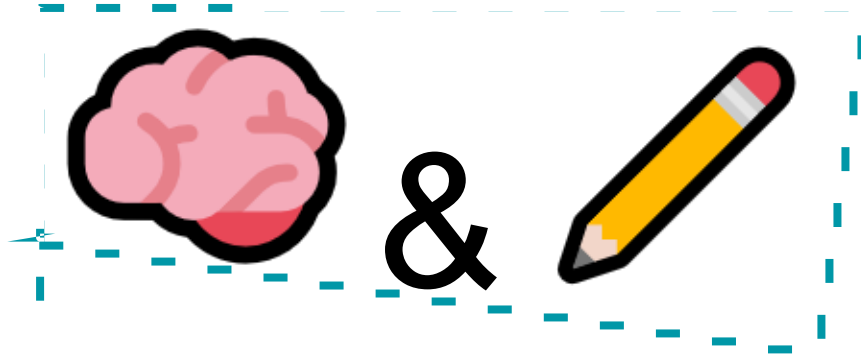
- Identify the descriptive statistics associated with the 1998 fish data set

## Run code to identify the mean amount of mercury

```
290 **What is the mean (average) amount of mercury found in local fish? **
291
292 ```{r mean amount of mercury}
293 # Below, we will use "$" to select the column called "mercury" in the data
294 # If we wanted the mean fish length, we;d have written mean(fish_1998$length)
295
296 mean(fish_1998$mercury)
297
298 ```
299
300 The "mean" is the average level of mercury in the data.
```

```
[1] 0.4481177
```





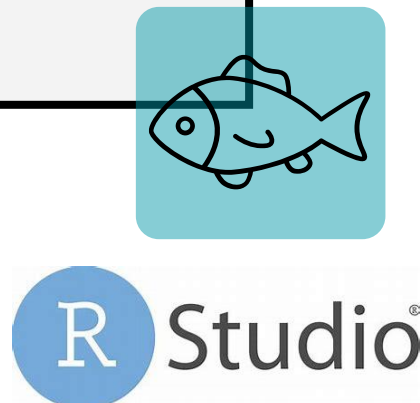
What is the mean level of mercury in the data?

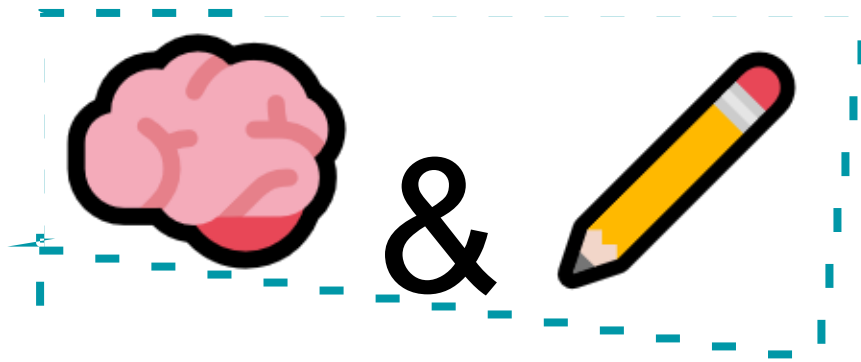
Why would knowing the mean (average) amount of methylmercury in fish be useful for fishery managers and the community?

## Run code to identify the median amount of mercury

```
305
306 **The median**
307
308 The "median" is the middle level of mercury data (half the fish in the dataset have more
mercury and half have less).
309
310 * Means and medians are often similar, However:
311 * They will be different if the data have a lot of large values or a lot of small valued
(skewed data).
312 * Sometimes you are asked to use one or the other, so it's good to be careful about
which you are calculating.
313
314 ```{r median amount of mercury}
315 # remember, in the last code chunk, we used mean(fish_1998$mercury)
316 # The code below is similar, but it calculates the median.
317
318 median(fish_1998$mercury)
319 ```
```

```
[1] 0.4481177
```





What is the median level of mercury in the data?

How would we interpret these median and mean values? Should the K'avi community be concerned?

# Find the minimum and maximum amount in the data set

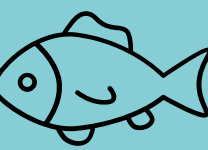
```
329 Work in groups to see if you can find these values using the commands max() and min()
330
331 For help, you can google "R min()" or you can type ?min() in the "Console" window below,
 then read the help file in the "Output" window on the lower right.
```

```
332
333 {r max and minimum amount of mercury}
334 # Type your code below to calculate the mininum and maximum values of methylemercury in
 the 1998 fish dataset.
335
336 min(fish_1998$mercury)
337
338
```

```
333 {r max and minimum amount of mercury}
334 # Type your code below to calculate the mininum and maximum values of methylemercury in
 the 1998 fish dataset.
```

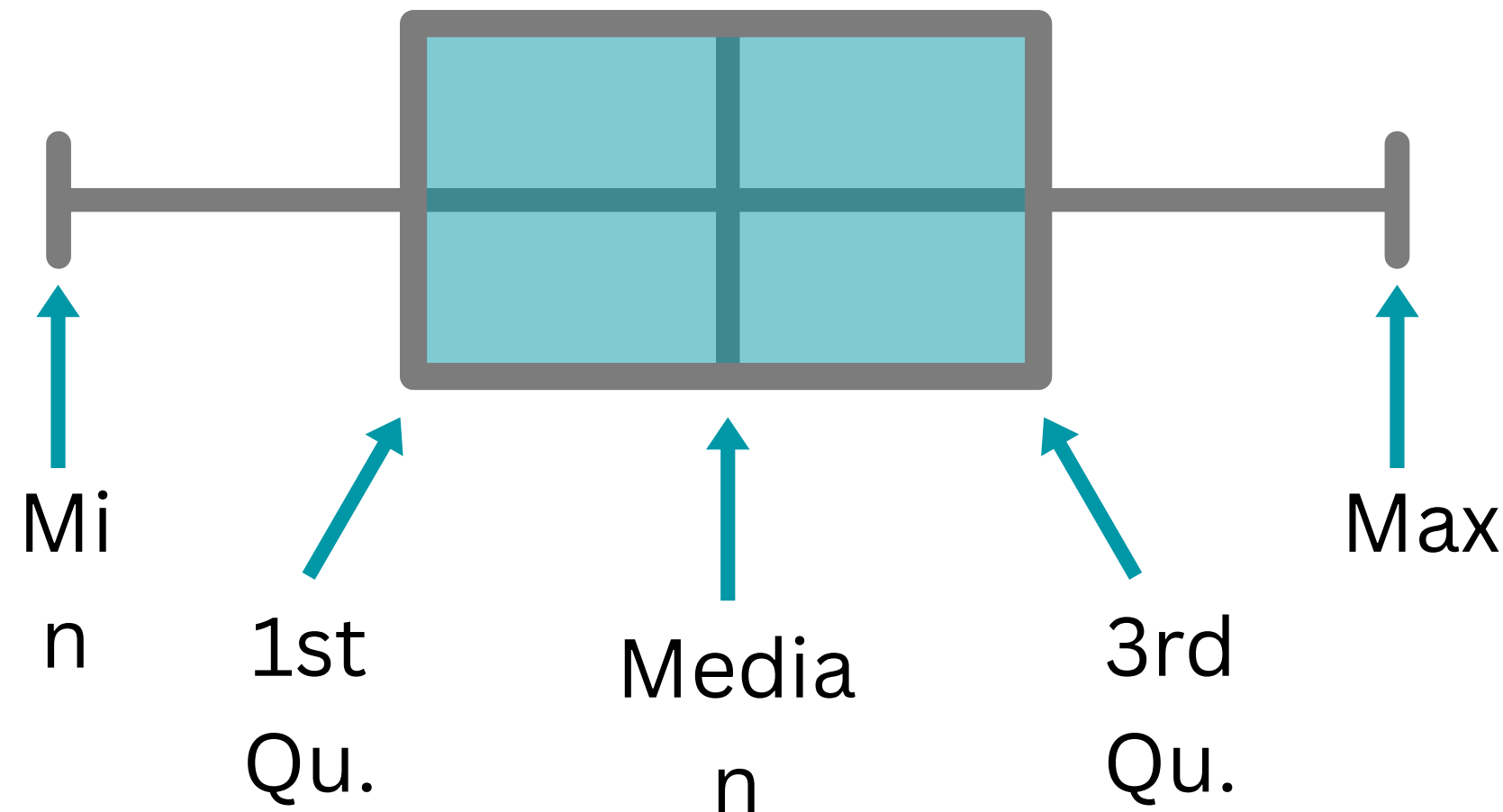
```
335
336 max(fish_1998$mercury)
337
338
```

```
[1] 1.894938
```



# Descriptive Statistics

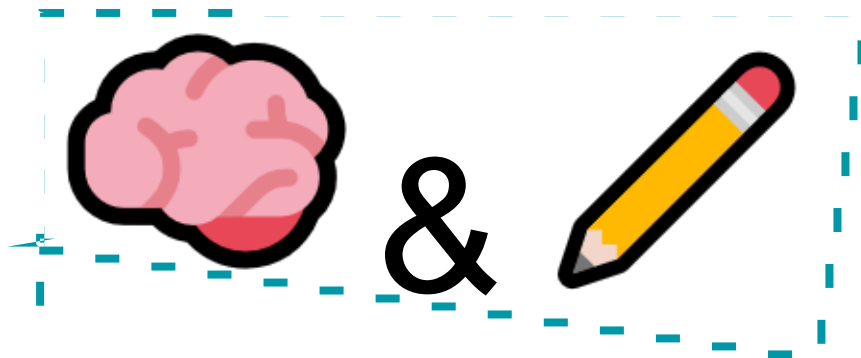
- Provide key information about the distribution of data
- The mean, median, minimum and maximum are all "descriptive statistics"
  - 1st Qu. and 3rd Qu. refer to the 1st and 3rd "Quartiles", which represent a quarter of the data
- The function `summary()` estimates all these descriptive statistics



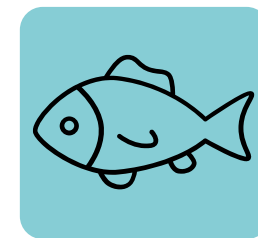
## Run code to generate summary statistics

```
349 {r summary stats}
350 # The format of this summary() command is similar to other commands.
351 summary(fish_1998$mercury)
352
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
0.007764	0.278103	0.413540	0.448118	0.557474	1.894938



Looking at all of these values, are there unsafe levels of mercury in local fish? Are there dangerous levels of mercury in most fish, or only some?



# Lesson 5: Working with ggplot to Generate a Graph of Mercury Concentrations

## Goals:

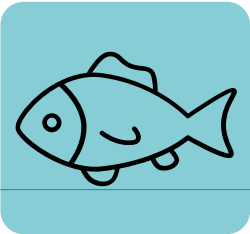
- Create a scatterplot and boxplot using the ggplot2 package to visualize the distribution of mercury concentration levels
- Create a scatterplot using the ggplot2 package to visualize the potential relationship between fish length and mercury concentrations
- Analyze the distribution of mercury in fish to determine how often fish have mercury levels above the EPA recommended safe level



[ggplot2 cheatsheet](#)

# Plotting Distribution of Mercury using ggplot2 functions

Code	What is it doing?
<code>ggplot (fish_1998, aes(x= 'fish', y= mercury))+</code>	<code>aes()</code> tells ggplot what we want on each axis X = fish Y = mercury   Note: by calling x “fish”, it is not a variable and we will look at variations in length independent of any other variable
<code>geom_jitter(width = 0.1)+</code>	<code>Geom_jitter</code> tells ggplot to make a jitter plot   Width = 0.1 tells ggplot to specify the width of the points to this size
<code>geom_hline(yintercept = 0.46, linetype= "dashed", color= "red", size=1)+</code>	This tells ggplot to create a dashed red line that is a specific size as a y-intercept at the EPA threshold safe level; then we can see how many fish are beyond it.
<code>xlab("fish")+ ylab("mercury (ug/g))+ ggtitle("1998 Mercury Distribution")+</code>	Labels the x axis, y axis, and provides a title
<code>theme_minimal()</code>	adjusts the non-data components of the plot, so the titles, labels, background, and legends among other aspects

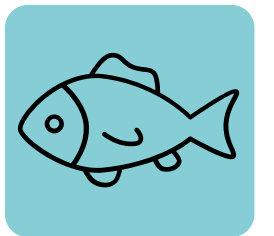


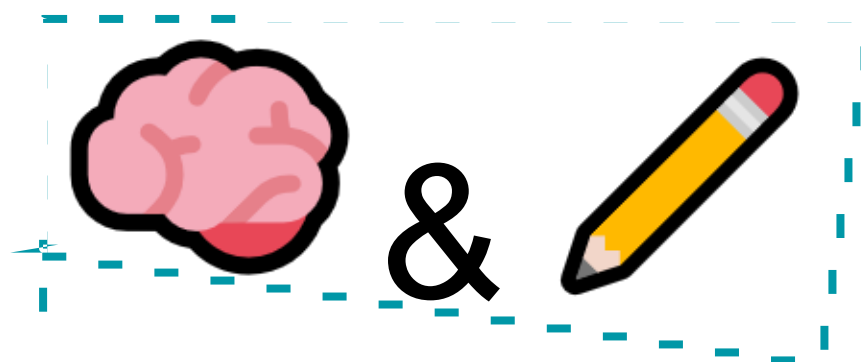
# Run code to plot the distribution of mercury levels

Warning: Using  
`size`  
aesthetic for  
lines was  
deprecated in  
ggplot2 3.4.0.  
Please use  
`linewidth`  
instead.

I got this result after  
running the code.

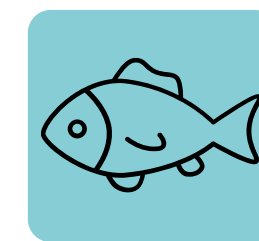
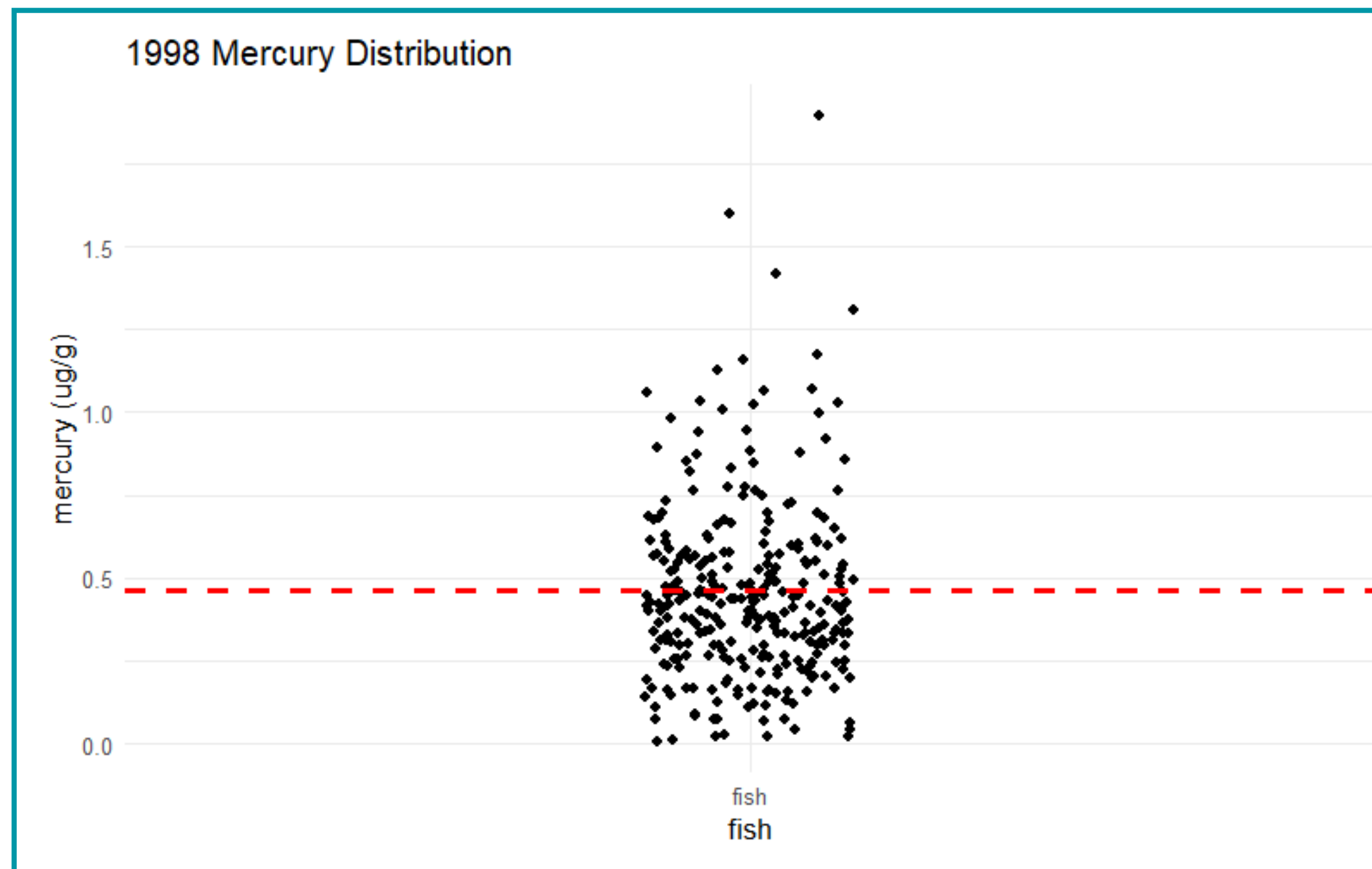
```
379 - ### Plot distribution of 1 variable
380
381 - ```{r ggplot, Ribbon Plot, mercury}
382 #For this plot, we want to see the distributions of mercury in the fish and how many of
 them are above the EPA's recommended levels
383 #ggplot is the function that we are using to create a plot. aes() is used to tell ggplot
 what we want to see on each axis, in this case x= 'fish' and y= mercury. x="fish"
 indicates that the x axis doesn't have a variable and is titled fish. This allows us to
 look at the variation in lengths of fish independent of another variable.
384 ggplot (fish_1998, aes(x= 'fish', y= mercury))+
385 #Since ggplot is just a function to create plots, we need to further specify what kind of
 plot it should create. We can do this by using the geom_jitter function, which will create
 the jitter plot seen below. We can also add a few more parameters to the graph, such as
 changing the width of the points (width = 0.1).
386 #The jitter plot is a variation on the scatterplot, but it separates the data so that we
 can distinguish how many points are above the unsafe mercury level. To see the difference,
 try geom_point() and then geom_jitter()
387 geom_jitter(width = 0.1)+
388 #We can also add shapes to the plot. Here we added a line to show where the EPA's
 guideline for unsafe levels of mercury in fish would fall. To create this we used the
 geom_hline() function which simply creates a horizontal line at a y-intercept that you
 set. We can also adjust the color and appearance of the line with color = 'red'.
389 geom_hline(yintercept = 0.46, linetype= "dashed", color= "red", size=1)+
390 #This allows us to relabel the x-axis
391 xlab("fish")+
392 #This allows us to relabel the y-axis
393 ylab("mercury (ug/g)")+
394 #This allows us to title the graph
395 ggtitle("1998 Mercury Distribution")+
396 #This adjusts the non-data components of the plot, so the titles, labels, background, and
 legends among other aspects.
397 theme_minimal()
398 ```
399 -
```





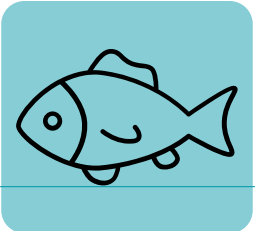
What does this distribution tell us about the mercury levels in the fish?

Look back at the min, max, median, and mean values that were calculated above. Where would these values fall on this graph? Are the mean and median values above or below the level suggested by the EPA?



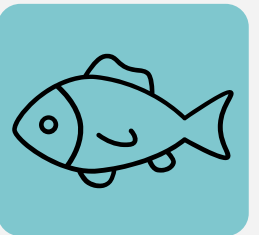
# Add a Boxplot on top of the distribution plot–the highlighted text is what is different from our first plot

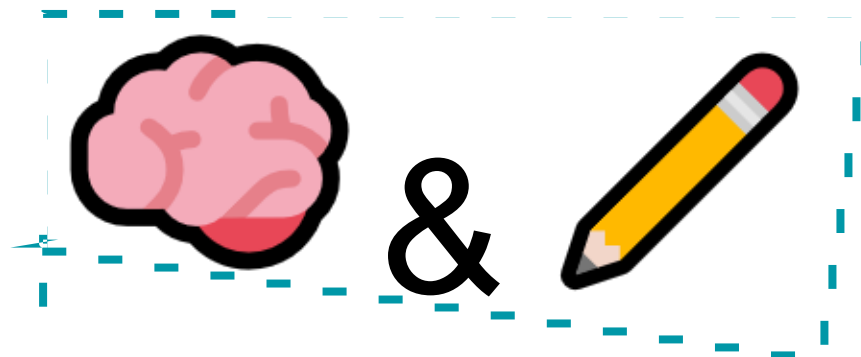
Code	What is it doing?
<code>ggplot (fish_1998, aes(x= 'fish', y= mercury))+</code>	<code>aes()</code> tells ggplot what we want on each axis X = fish Y = mercury   Note: by calling x “fish”, it is not a variable and we will look at variations in length independent of any other variable
<code>geom_jitter(width = 0.1)+</code>	<code>Geom_jitter</code> tells ggplot to make a jitter plot   Width = 0.1 tells ggplot to specify the width of the points to this size
<code>geom_boxplot(alpha=0.5, color="dodgerblue2")+</code>	<code>Geom_boxplot</code> overlays a boxplot on the distribution Alpha changes the opacity so we can see the distribution plot below it.
<code>geom_hline(yintercept = 0.46, linetype= "dashed", color= "red", size=1)+</code>	This tells ggplot to create a dashed red line that is a specific size as a y-intercept at the EPA threshold safe level; then we can see how many fish are beyond it.
<code>xlab("fish")+ ylab("mercury (ug/g)")+ ggtitle("1998 Mercury Distribution + Box Plot")+</code>	Labels the x axis, y axis, and provides a title
<code>theme_minimal()</code>	adjusts the non-data components of the plot, so the titles, labels, background, and legends among other aspects



## ▶ Run code to add a boxplot on top of the distribution plot

```
410 `{{r, Ribbon and Box Plot}}
411 # These lines are almost the same as the last code chunk,
412 # but we added a '+' at the end, to add some new pieces to the graph.
413 ggplot (fish_1998, aes(x= 'fish', y= mercury))+
414 geom_jitter(width = 0.1)+
415 # We will overlay a boxplot on the jitter plot.
416 # We can adjust its color as we did above,
417 # We use alpha, to adjust the opacity of the boxplot.
418 # This lets us see the jitter plot beneath it.
419 geom_boxplot(alpha=0.5, color="dodgerblue2")+
420 # As before, we add a line to represent the EPA's recommended safe mercury ingestion level
421 geom_hline(yintercept = 0.46, linetype= "dashed", color= "red", size=1)+
422 # The rest of the code is the same as the last code chunk:
423 xlab("fish")+
424 ylab("mercury (ug/g)")+
425 ggtitle("1998 Mercury Distribution + Box Plot")+
426 theme_minimal()
427 `{{`
```



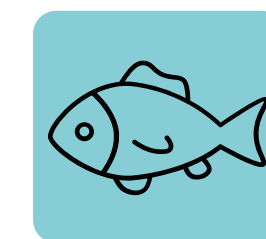
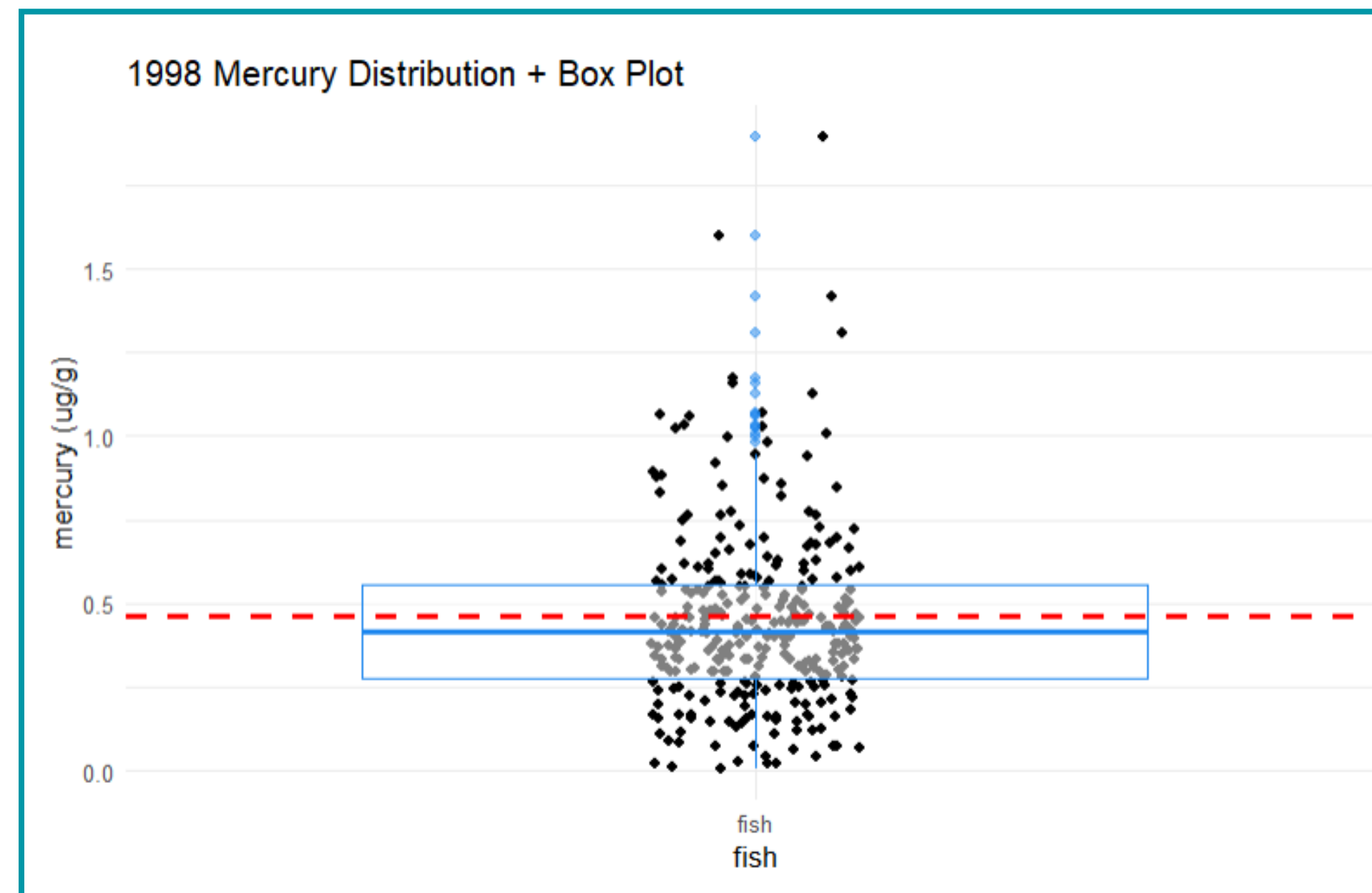


From looking at this graph, are there unsafe levels of mercury in local fish?

If so, is it in most fish, or only some? Did this graph tell you anything different than the summary statistics, or did it just tell you the same information in a different way?

In 1998, was it safe for members of the K'avi community to eat fish from these waters?

How comfortable would you have been in 1998 about eating a fish that you caught?



# Now, Let's Plot Two Variables

## Mercury and Fish Length

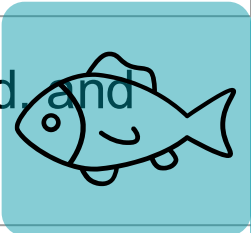
- Mercury builds up in fish as they age
- Analyze the relationship between fish size and fish mercury levels to give us useful guidance for which fish were safe to eat back in 1998



[Image Source](#): ThermoFisher

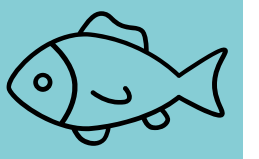
# Create a scatterplot

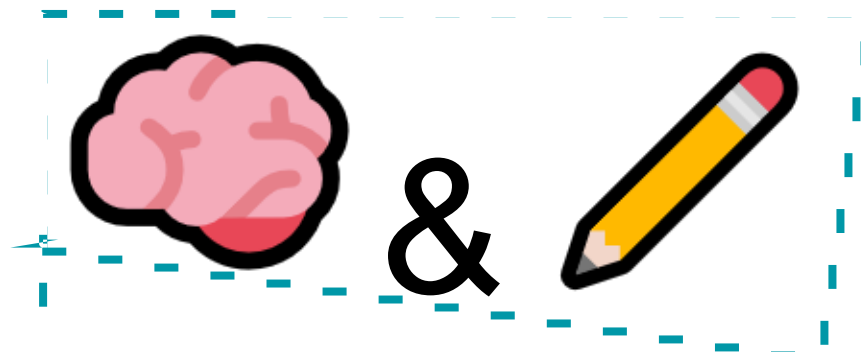
Code	What is it doing?
<pre>LengthMercury1998 &lt;- ggplot(fish_1998, aes(x = length, y = mercury)) +</pre>	LengthMercury1998 defines the plot with the details after it   Remember, &lt;- is like =   Length will be on the x-axis (horizontal) Mercury will be on the y-axis (vertical)
<pre>geom_point(color = 'blue', size = 2) +</pre>	geom_point tells ggplot to make a scatter plot   Color will make the dots blue and the size of 2 is a little bigger than the default
<pre>xlab("length (cm)")+ ylab("mercury (ug/g)")+ ggtitle("Mercury (ug/g) vs. Length (cm) (1998)")+</pre>	Labels the x axis, y axis, and provides a title
<pre>geom_hline(yintercept = 0.46, linetype= "dashed", color= "red")+</pre>	This tells ggplot to create a dashed red line that is a specific size as a y-intercept at the EPA threshold safe level; then we can see how many fish are beyond it.
<pre>theme_minimal()</pre>	adjusts the non-data components of the plot, so the titles, labels, background, and legends among other aspects
<pre>LengthMercury1998</pre>	Because we assigned the graph a label, we have to call that label



## ▶ Run code to create a scatterplot for length of fish and mercury concentration

```
446 `r`{r ggplot, Day1 1998 Scatter Plot}
447 # This line defines a plot, called 'LengthMercury1998', as a ggplot().
448 # The ggplot() code is a little different from the single variable boxplot.
449 # Now, we are plotting two columns of the dataset:
450 # length will be on the (horizontal) x-axis
451 # mercury will be on the (vertical) y-axis
452 LengthMercury1998 <- ggplot(fish_1998, aes(x = length, y = mercury)) +
453 # geom_point() makes a scatterplot.
454 # We are going to plot the data points in blue
455 # and a little bigger than the default "size = 2"
456 geom_point(color = 'blue', size = 2) +
457 # Now we're going to write labels for the two axes, and give the graph a title
458 xlab("length (cm)") +
459 ylab("mercury (ug/g)") +
460 ggtitle("Mercury (ug/g) vs. Length (cm) (1998)") +
461 # As before, we will add a line to represent the EPA's recommended mercury level
462 # This time, we'll make a red dashed line.
463 geom_hline(yintercept = 0.46, linetype = "dashed", color = 'red') +
464 # We'll use the same theme as last time.
465 theme_minimal()
466
467
468 #Because we assigned the graph a label, we have to call that label
469 LengthMercury1998
470 `r`
```

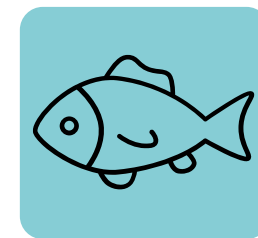
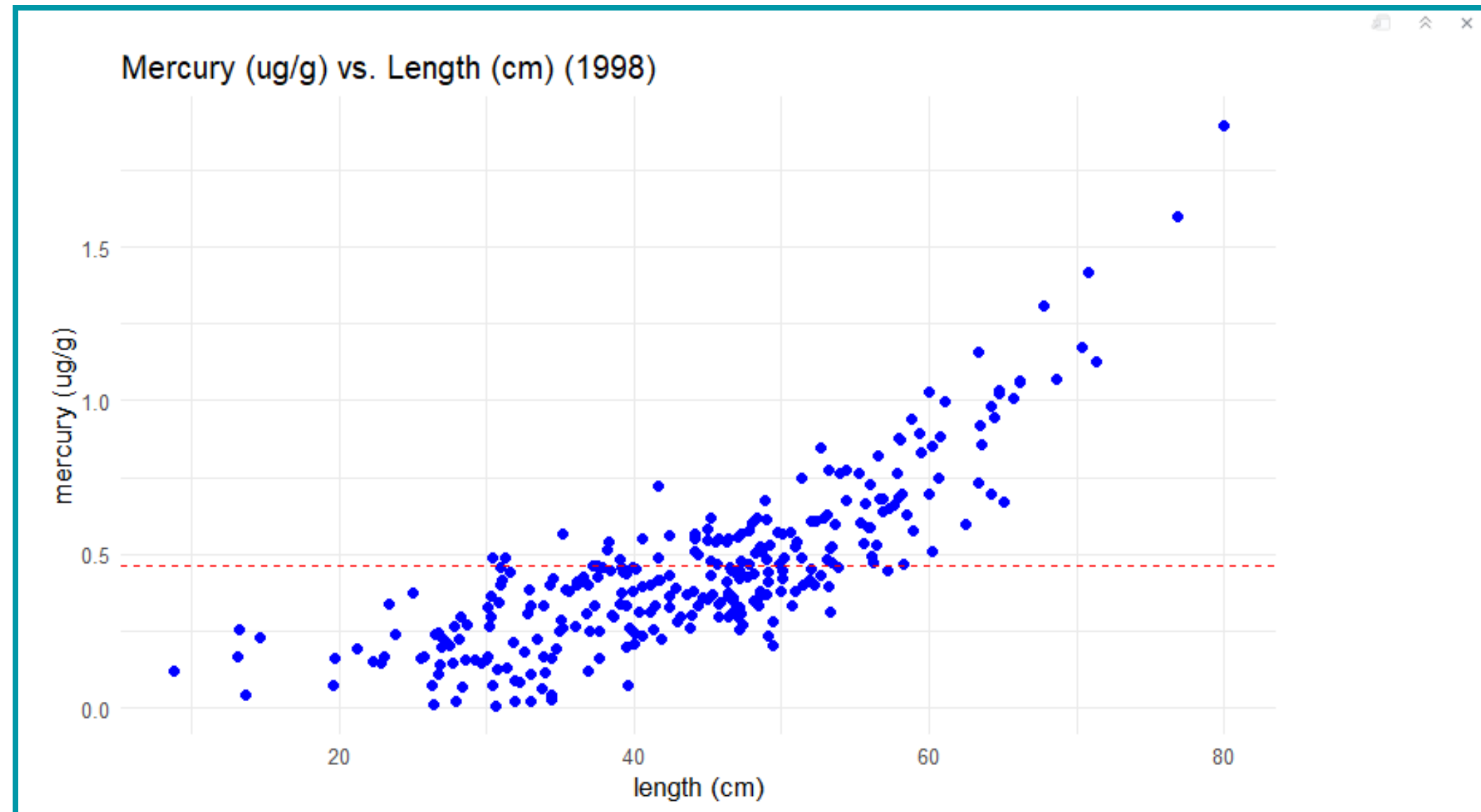




What does this tell us about the relationship between fish length and mercury levels?

Are there certain lengths of fish that have a higher amount of mercury?

If you caught a fish in 1998, would this graph help you decide whether the fish was safe?





Day

1



Day

2

# Lesson 6: Load in and Explore 2023 Fish Data

The 1998 results showed health hazards from fish with high methylmercury motivated the tribe to undertake a project to remove contaminated sediments.

In 2023, 50 years after the initial measurements, K'avi fisheries managers conducted another survey of mercury levels in fish.

They carefully collected data from the same sites they surveyed in 1998.



# Lesson 6: Load in and Explore 2023 Fish Data

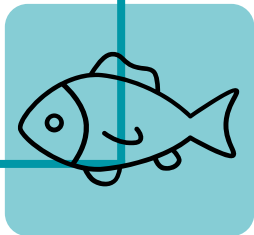
## Goals:

- Load in the 2023 fish data set
- Identify code to run in head() function
- View the entire data set in spreadsheet form
- Identify code to run in the dim() function
- View a truncated version of the data set
- Identify the number of rows and columns in the 2023 fish data set



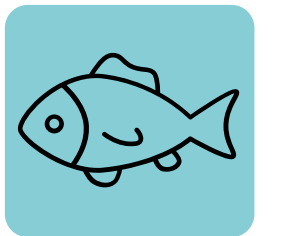
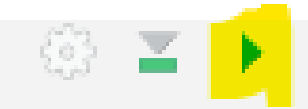
# Load “tidyverse” package; Install and load “patchwork” package

Package	What does it do?	What functions are we using/learning?
tidyverse	used for 'tidying' data: wrangling, analysis, visualizing	select, filter, mutate, ggplot, summarize
patchwork	Allows plots to be viewed side by side	

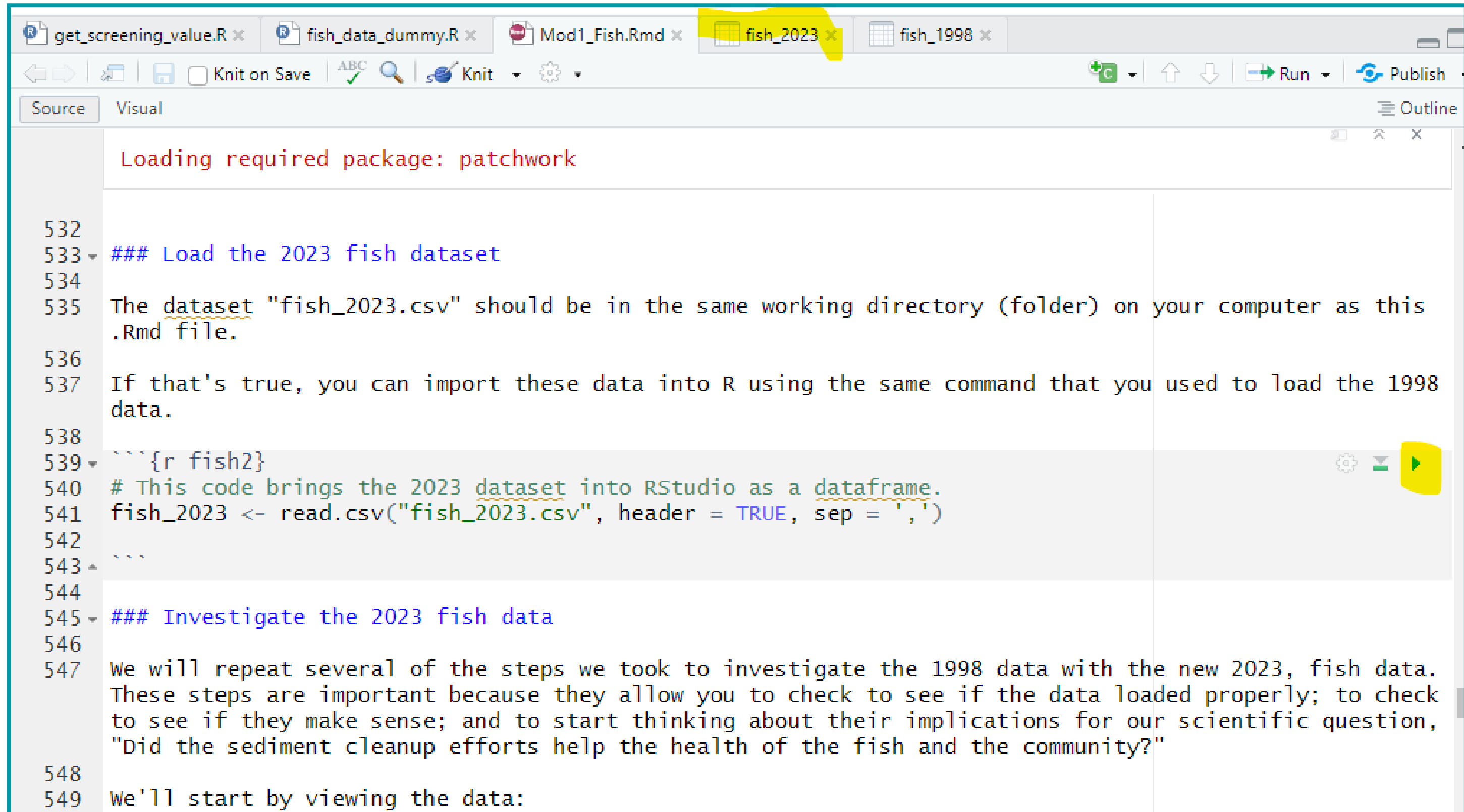


# Load “tidyverse” package; Install and load “patchwork” package

```
522 {r load packages2}
523 # This loads the "tidyverse" package that you've already installed.
524 library(tidyverse)
525
526 # This installs "patchwork"
527 if(!require("patchwork")) install.packages("patchwork")
528
529 # This loads patchwork into the library, so you can use it in this program.
530 library(patchwork)
531 }
```

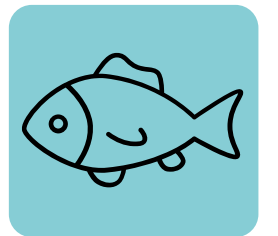


# ▶ Run Code to Load in 2023 Fish Data as a Dataframe



The screenshot shows the RStudio interface with several open files: `get_screening_value.R`, `fish_data_dummy.R`, `Mod1_Fish.Rmd`, `fish_2023` (highlighted in yellow), and `fish_1998`. The `fish_2023` file is active, showing R code in the Source pane. The code includes a comment about loading the 2023 fish dataset, instructions on where the `fish_2023.csv` file should be located, and the actual R code to read the CSV file into a dataframe named `fish_2023`. The code uses `read.csv` with `header = TRUE` and `sep = ','`. Below the code, there is a comment about investigating the 2023 fish data and a final line stating that the next step is to view the data.

```
532
533 ### Load the 2023 fish dataset
534
535 The dataset "fish_2023.csv" should be in the same working directory (folder) on your computer as this
536 .Rmd file.
537 If that's true, you can import these data into R using the same command that you used to load the 1998
538 data.
539 ```{r fish2}
540 # This code brings the 2023 dataset into RStudio as a dataframe.
541 fish_2023 <- read.csv("fish_2023.csv", header = TRUE, sep = ',')
542 ```
543
544
545 ### Investigate the 2023 fish data
546
547 We will repeat several of the steps we took to investigate the 1998 data with the new 2023, fish data.
548 These steps are important because they allow you to check to see if the data loaded properly; to check
549 to see if they make sense; and to start thinking about their implications for our scientific question,
550 "Did the sediment cleanup efforts help the health of the fish and the community?"
551
552 We'll start by viewing the data:
```



# Practice using the head() function

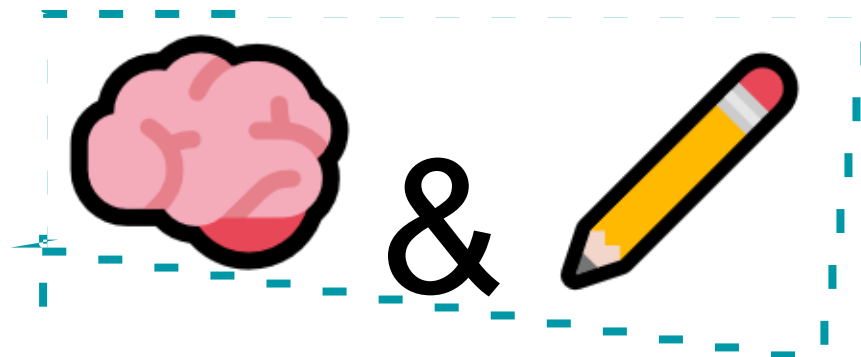
What would go in the parentheses of head() to see the first 6 rows?

```
5 558 {r View data pt. 2 - take 2, eval=FALSE}
5 559 #Fill in the parentheses:
5 560 head(fish_2023)
5 561
5 562
```

Description: df [6 × 3]

	X <int>	length <dbl>	mercury <dbl>
1	1	40.16618	0.1993075
2	2	37.33182	0.2296922
3	3	40.93916	0.2300245
4	4	75.38346	0.9531401
5	5	50.42610	0.2909152
6	6	66.00687	0.6845460

6 rows



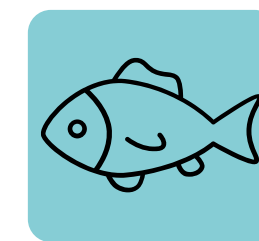
What are the variables that this data set contains?

```
558 {r View data pt. 2 - take 2, eval=FALSE}
559 #Fill in the parentheses:
560 head(fish_2023)
561
562
```

Description: df [6 × 3]

	<b>X</b> <int>	<b>length</b> <dbl>	<b>mercury</b> <dbl>
1	1	40.16618	0.1993075
2	2	37.33182	0.2296922
3	3	40.93916	0.2300245
4	4	75.38346	0.9531401
5	5	50.42610	0.2909152
6	6	66.00687	0.6845460

6 rows



# Practice using the dim() function

What would go in the parentheses of dim() in order to identify the number of rows and columns the 2023 dataset?

```
570 {r_explore_data_dimensions 2, eval=FALSE}
571 #Fill in the parentheses with the name of the dataframe for the 2023 fish data
572 dim(fish_2023)
573
574
```

```
[1] 300 3
```

# Lesson 7: Identify the Descriptive Statistics of the 2023 Dataset

According to the EPA, the maximum safe level of mercury to be consumed is around 0.46 ug/g/day. now that the rivers have been cleaned, tribal fishery managers are interested in how mercury levels in fish belly fat have changed from 1998 to 2023.



## Goals:

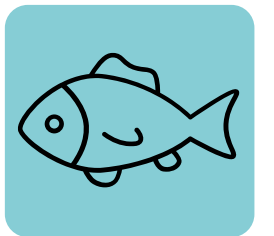
- Identify the descriptive statistics associated with the 2023 fish data set
- Analyze data points from the 2023 fish data set to answer the following questions for tribal fishery managers:
  - Do the mercury levels in local fish belly fat exceed the maximum safe level of mercury to be consumed?

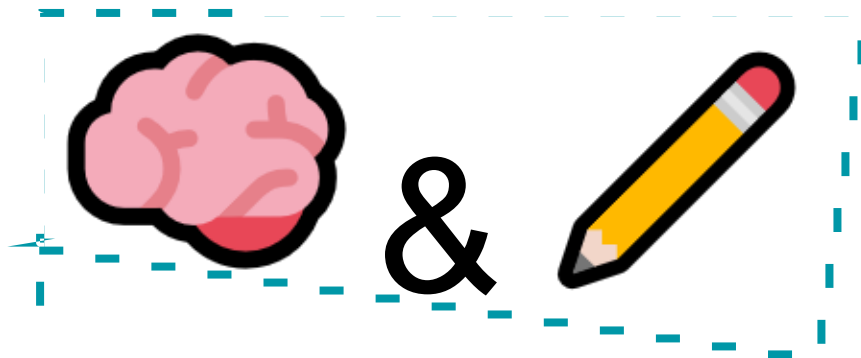
# Descriptive Statistics Code for 2023 Fish Data

Work together to write code that tells you the descriptive statistics (median, mean, minimum, maximum, etc) for the mercury in the fish collected in 2023.

```
596 {r mean amount of mercury 2}
597 # write code for summary statistics here
598 summary(fish_2023$mercury)|
599
600 ...
```

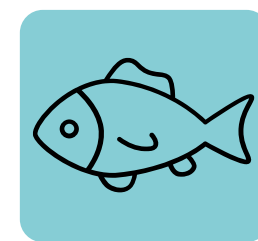
Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
0.002966	0.179127	0.277825	0.336867	0.423762	1.623886





How do the mean and median values of mercury in 2023 compare to the EPA's standard?

Do the descriptive statistics suggest that the mercury levels in fish have improved since 1998?



# Lesson 8: Making Graphs to Learn if the Size of Safe Fish Has Changed

## Goals:

- Create a plot of the relationship between mercury and fish length in 1998
- Create a plot of the relationship between mercury and fish length in 2023
- Plot the 1998 and 2023 side-by-side
- Analyze data points from the 2023 fish data set to answer the following questions for tribal fishery managers:
  - Do the mercury levels in local fish belly fat exceed the maximum safe level of mercury to be consumed?
  - Is there a relationship between the amount of mercury in fish and fish size?



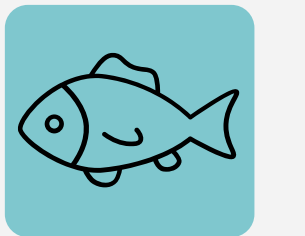
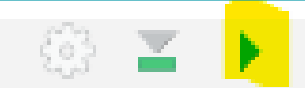
# Check Your Understanding

Look more carefully at the code this time. You will see things that look similar to the code you've run before. This is a good starting point for learning to make graphs in R. To test yourself, see if you can describe what each line of code does. What makes sense? What seems confusing?

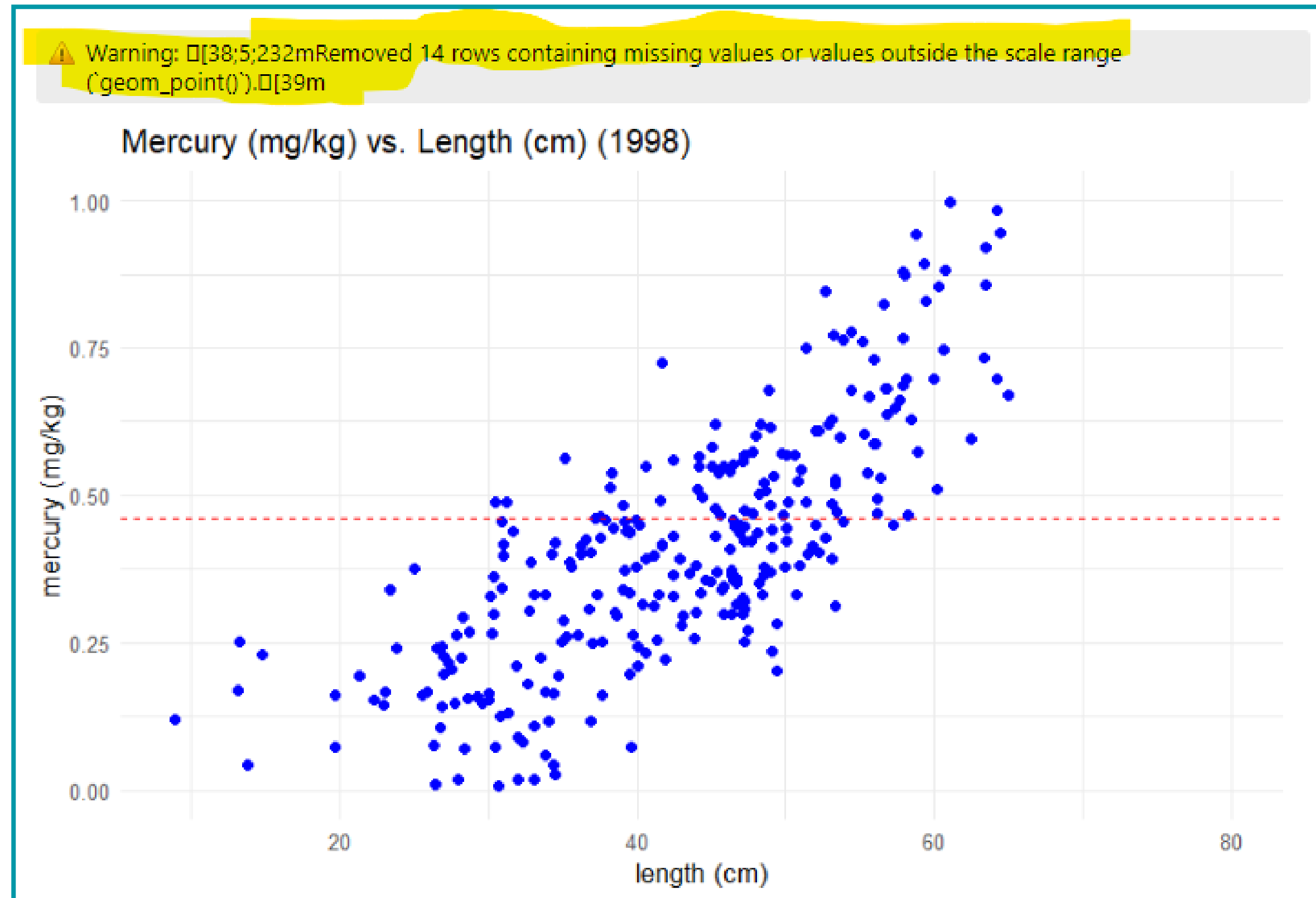


## ▶ Run code to make a scatterplot of 1998 data

```
631 # {r ggplot, Day2 Scatter Plot 1998 Fish}
632 # This code is the same code we used before.
633 # If you look at each line, what makes sense? What is confusing?
634 plot1998<- ggplot(fish_1998, aes(x = length, y = mercury)) +
635 geom_point(color = 'blue', size = 2) +
636 xlab("length (cm)") +
637 ylab("mercury (mg/kg)") +
638 ggtitle("Mercury (mg/kg) vs. Length (cm) (1998)") +
639 geom_hline(yintercept = 0.46, linetype = "dashed", color = 'red') +
640 # This is the only NEW line of code.
641 # We use the command ylim() to set the range of the y-axis
642 # The y-axis in this plot will start at 2 mg/kg mercury and end at 6 mg/kg
643 # We'll use the same range for the scatter plot of the 2023 data.
644 # That will make it easier to compare the fish across the two years.
645 ylim(0,1) +
646 # We will specify the same "minimal" theme we used before.
647 theme_minimal()
648
649 # Now let's plot the data from 1998
650 plot1998
651 #
```

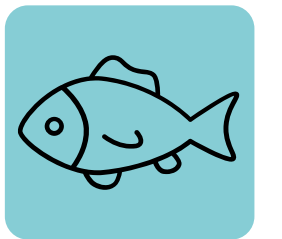


# Mercury vs. Length scatterplot 1998 data



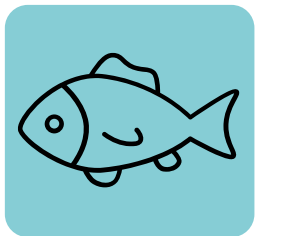
# Pro Tips on Becoming a Better Data Scientist

- Feel free to use code that worked before. A nice thing about coding is that if you've done something once, you then have the code written down for future use.
- However, you have to be careful that you modify the code for the new situation. In this case, we have to change all the references to 1998 so that they refer to the 2023 data. We should also rename all the objects we make (like the graph that is called plot2023), so that we don't accidentally replace the old objects with the same name.
- It's really useful to look through the code line-by-line, so that you can catch all these little mistakes. You can even have a friend look at your code to see if it makes sense to someone else.
- If you follow this advice, things that seem difficult now become easier soon!

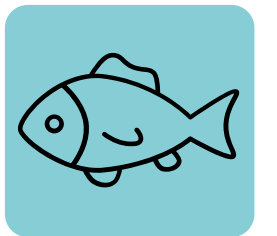
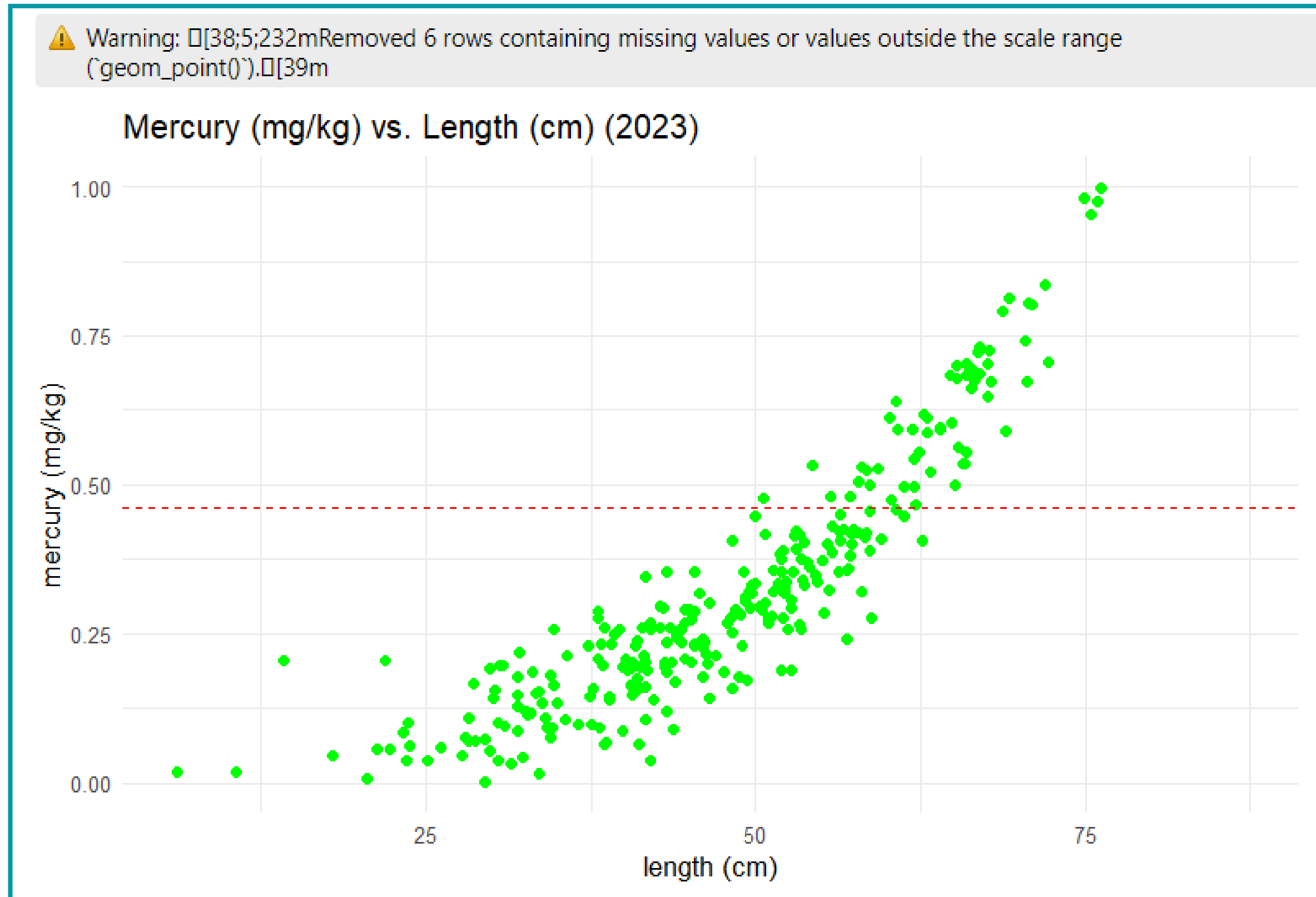


## ▶ Run code to make a scatterplot of 2023 data

```
663 ▶ ```{r ggplot, Scatter Plot 2023 fish}
664 # See if you can understand the code below. What changes have been made?
665 plot2023 <- ggplot(fish_2023, aes(x = length, y = mercury)) +
666 geom_point(color = 'green', size = 2) +
667 xlab("length (cm)") +
668 ylab("mercury (mg/kg)") +
669 ggtitle("Mercury (mg/kg) vs. Length (cm) (2023)") +
670 geom_hline(yintercept = 0.46, linetype= "dashed", color = 'red') +
671 # We use ylim() to match the y-axis here to that of plot1998
672 ylim(0, 1) +
673 theme_minimal()
674
675 plot2023
676 ▶ ```
```



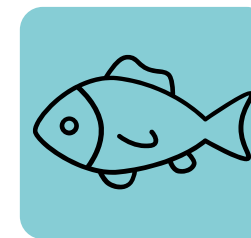
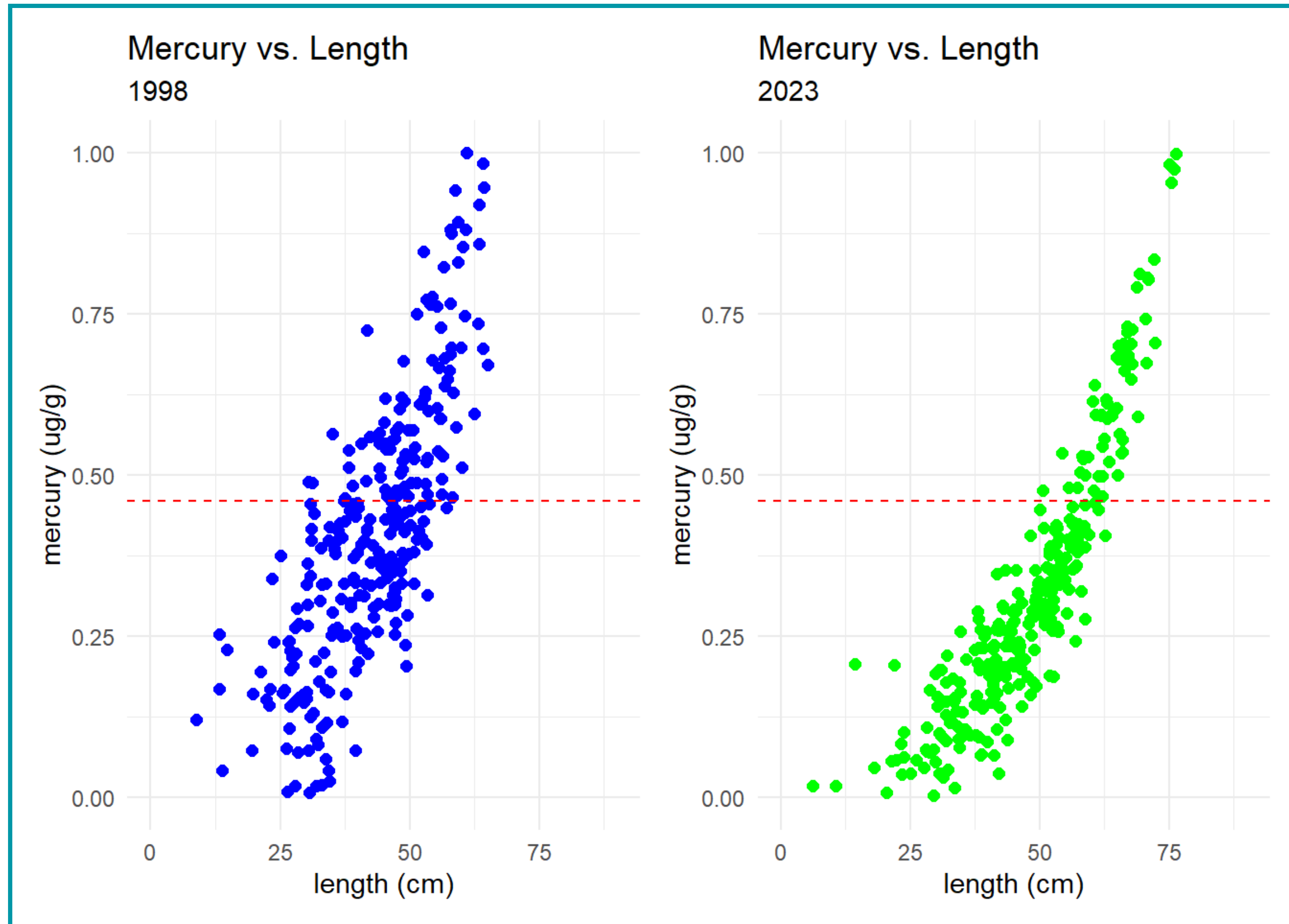
# Mercury vs. Length scatterplot 2023 data

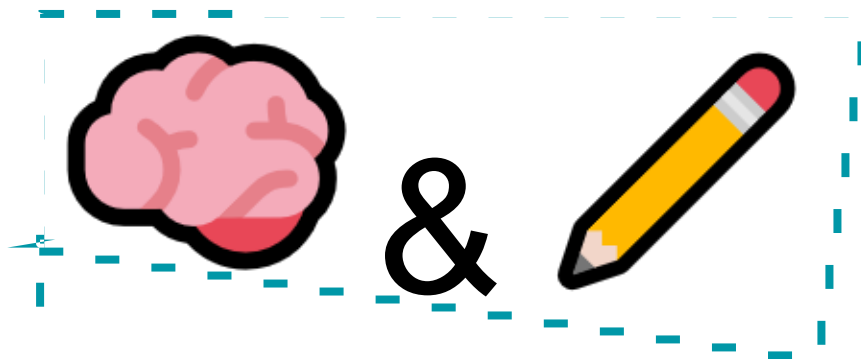


 Run code to plot 1998 next to 2023

```
682 ```{r putting plots side by side}
683 #plot from 1998 next to plot from 2023
684 plot1998 + plot2023
685 ```
```

# Side-by-side plot of 1998 and 2023 data

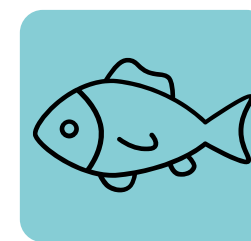




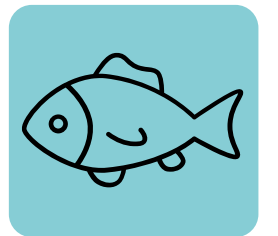
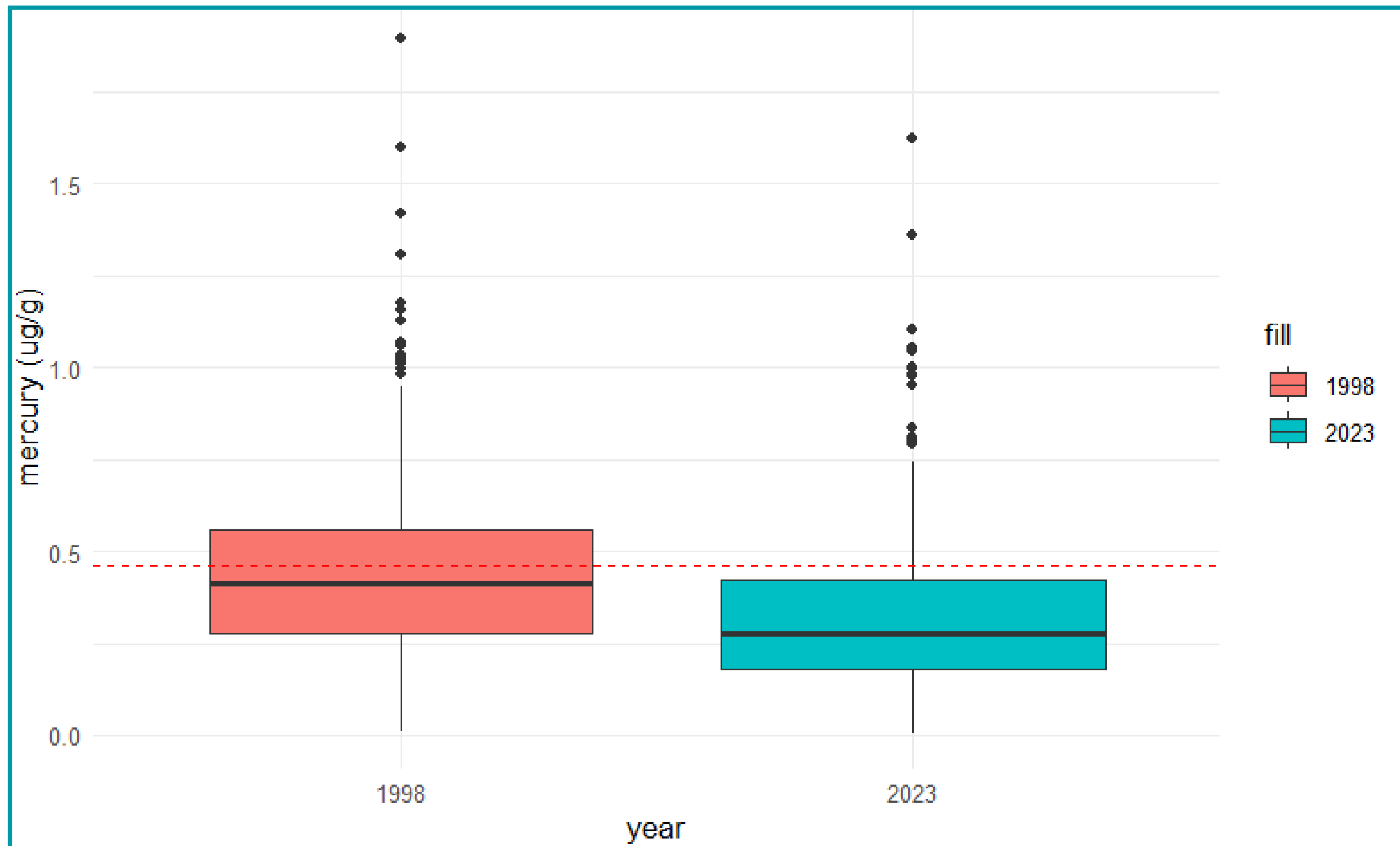
What is the safest large size fish to eat based on the 2023 dataset?  
How does this compare to the safest largest size fish that could be eaten based on the 1998 dataset?

Based on these plots, do you think the efforts since 1998 to clean mercury from the sediments were successful?

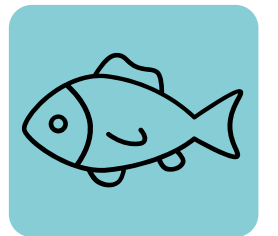
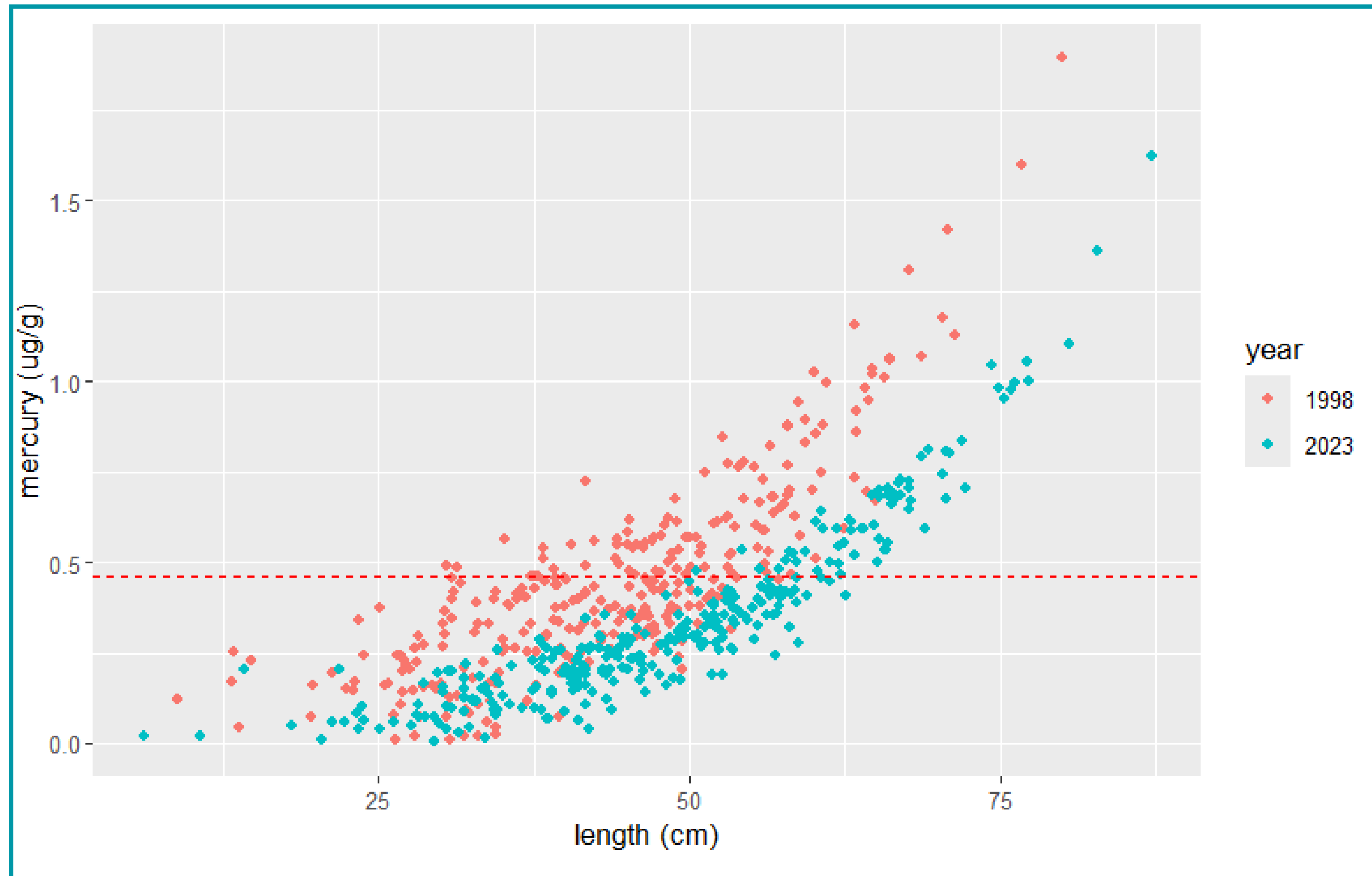
Work with a group to describe the evidence from these graphs for the success of the clean-up effort in a way that would be useful for the K'avi community.



# Optional Plot–Boxplot



# Optional Plot–scatterplot





*Day*

2

# Choose Your Path



[R and R studio](#)  
[Free, but requires](#)  
[downloading software](#)

[Best for:](#)  
[Laptops with memory](#)  
[Computer labs](#)



[binder](#)  
[Free, cloud-based service](#)  
[to run notebooks](#)

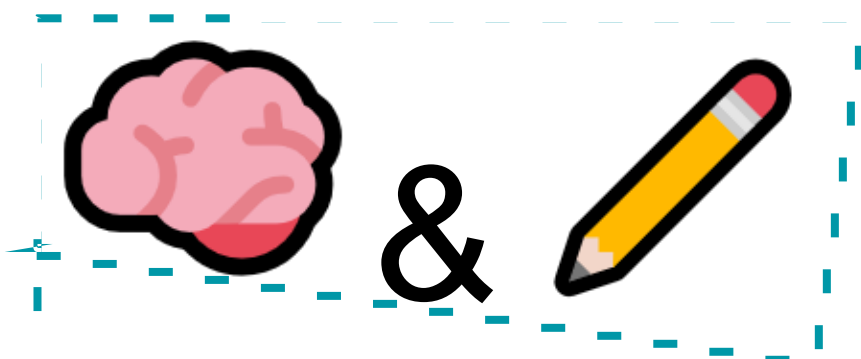
[Best for:](#)  
[Any environment](#)

# Creating Objects in R

An object is like a container or a box that holds data or information. You can give this box a name, and then use that name to look at or change what's inside the box later.

Example:

Imagine you want to store your age in R. You can create an object called `my_age` and put the number 25 in it.



Which objects were created to calculate consumption rate?

# Creating Objects in R--run the associated code chunks

## ▼ Creating objects in R

We will work with different types of objects and data types, including numbers, characters, vectors, and data frames. Here we start with creating objects for the variables and parameters used to calculate the screening value, or safe limit, of fish consumption. According to the [FDA](#), the screening value, or amount of safe fish consumption, depends on a person's body weight and how often they consume the fish.

First, the variables `weight` and `days_per_week` are set.

Once we create an object, we can use math or functions on it. Below we calculate `CR`, the mean daily consumption rate of the species of interest (g/d), based on the `days_per_week`

```
[3]: # Define variables
weight = 120 # lbs
days_per_week = 1 # days per week eating fish

print the values
print(weight)
print(days_per_week)

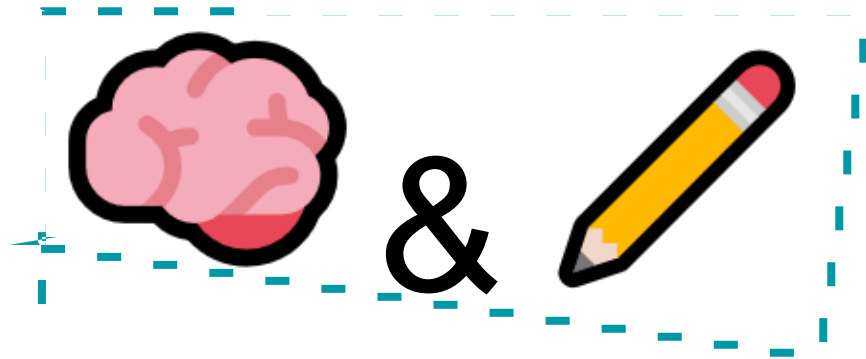
[1] 120
[1] 1

[4]: # use days per week to calculate mean daily consumption rate
cr = 110*days_per_week/7
print(cr)

[1] 15.71429
```

# Calculate Screening Values for Mercury in Fish

Run the code to determine the screening value for someone that is 120 lbs and eats fish 1 time per week.

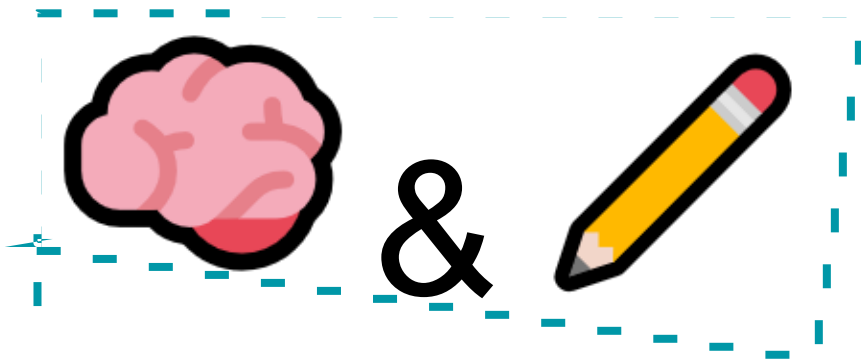


Change the value for weight or days per week, and record the changes to the safe limit on your student worksheet.

# Creating Vectors

A vector is a list of the same type of object (i.e., numbers, words, TRUE/FALSE).

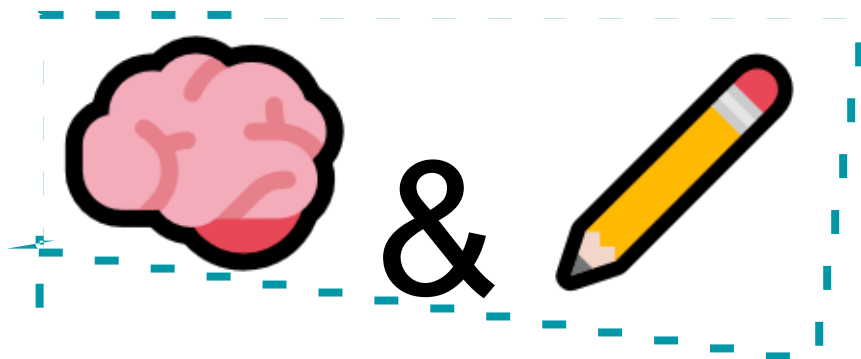
A vector is an object, but not all objects are vectors.



What function was used to  
create the vectors?

# Creating a Data Frame from Vectors

A data frame in R is a way to organize data in a table, similar to a spreadsheet. In a data frame, each column holds a group of related values (like weight, servings, and screening value), and each row represents one item or person.

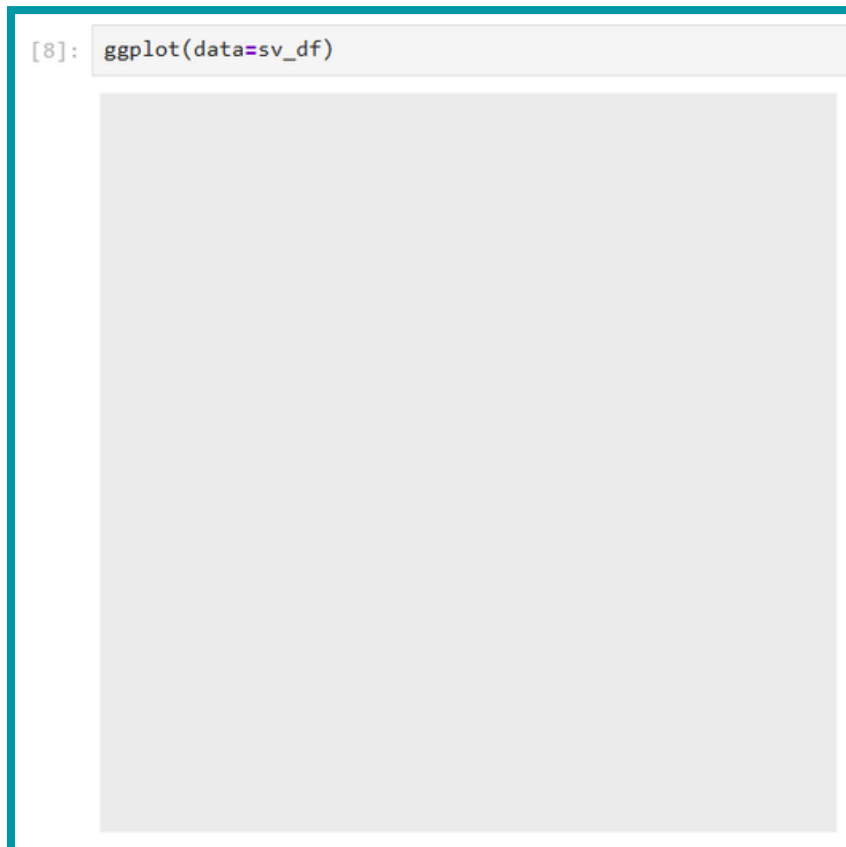


Provide an example of a time when you have made a spreadsheet.

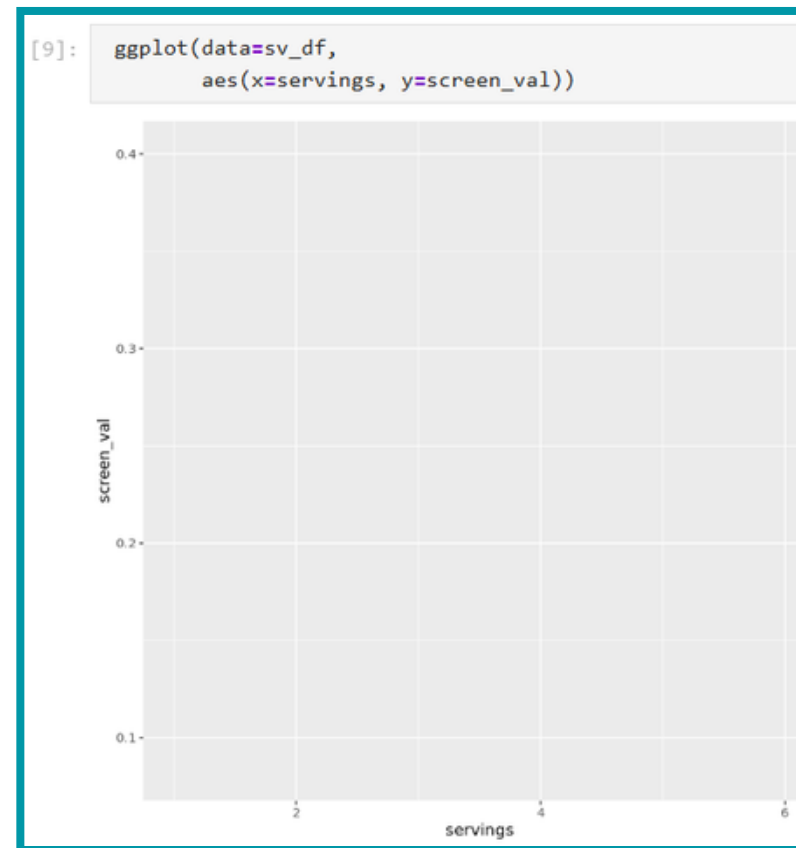
A data.frame: 3 × 3

	weight	servings	screen_val
	<dbl>	<dbl>	<dbl>
1	140	1	0.40090909
2	175	6	0.08352273
3	200	3	0.19090909

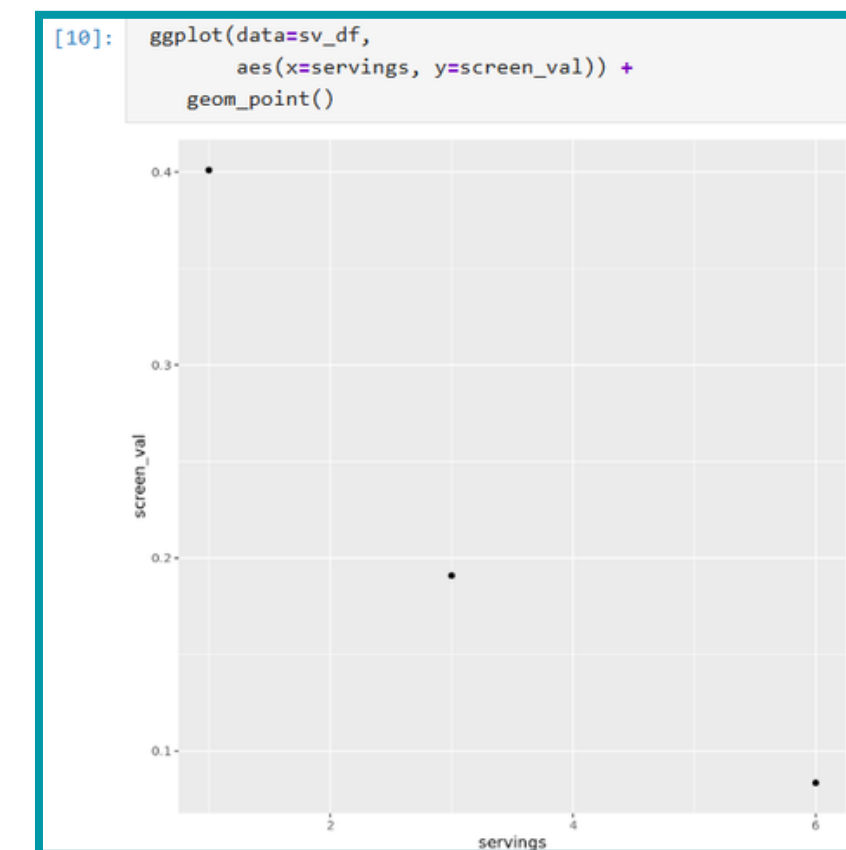
# Visualizing Screening Value vs. Servings Per Week with ggplot



Run ggplot with our data frame as the input

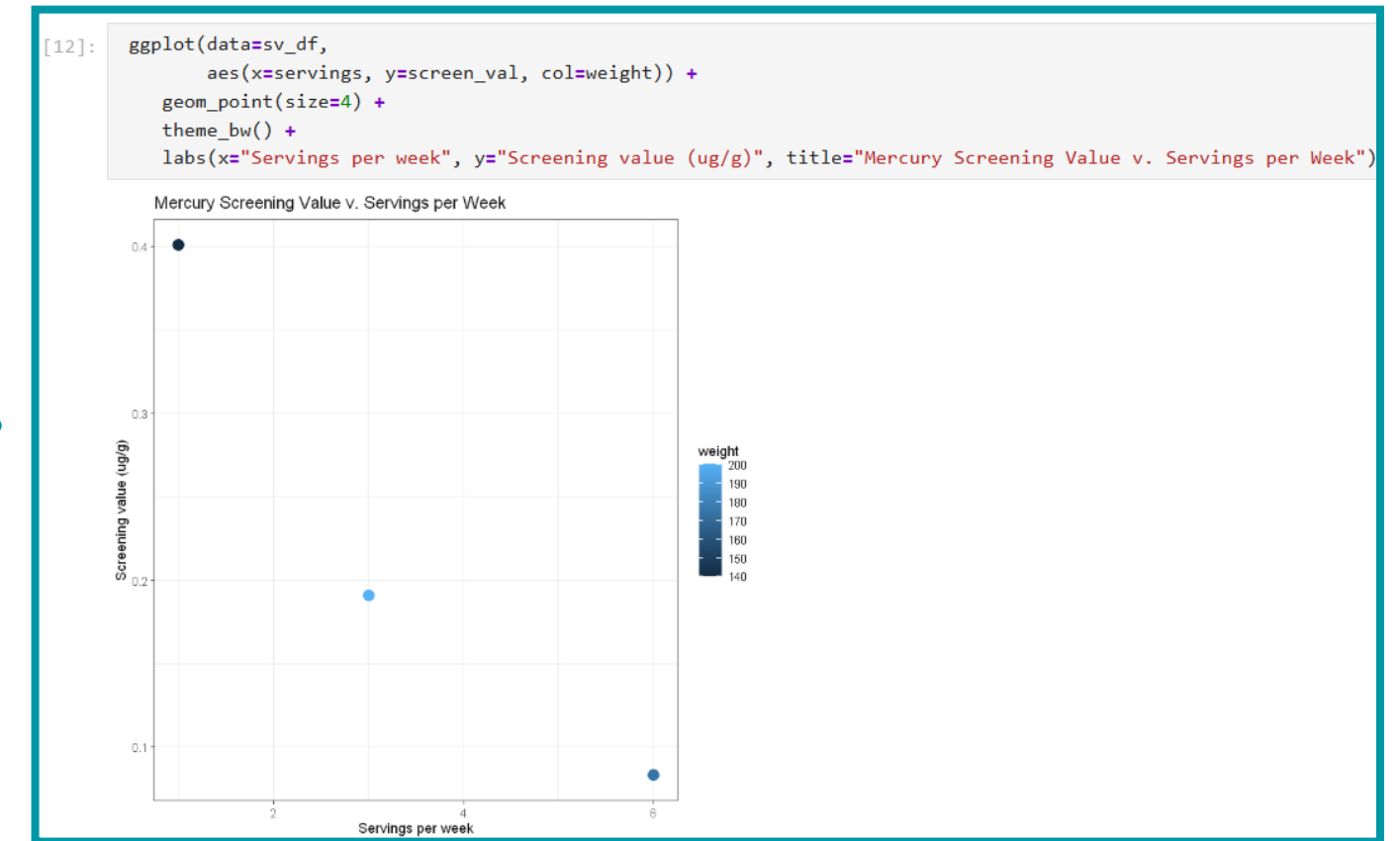
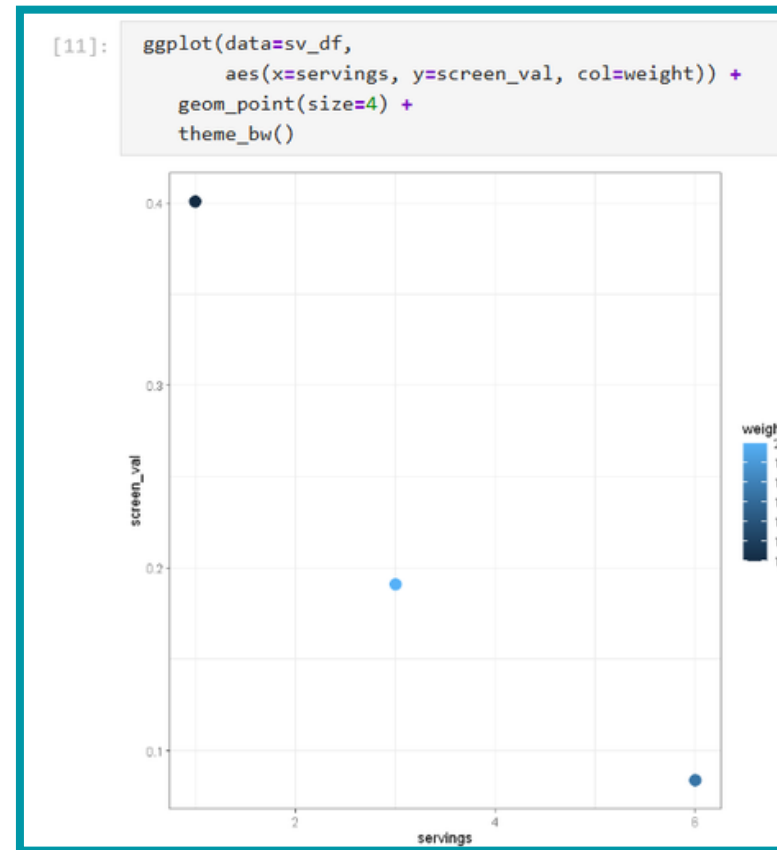
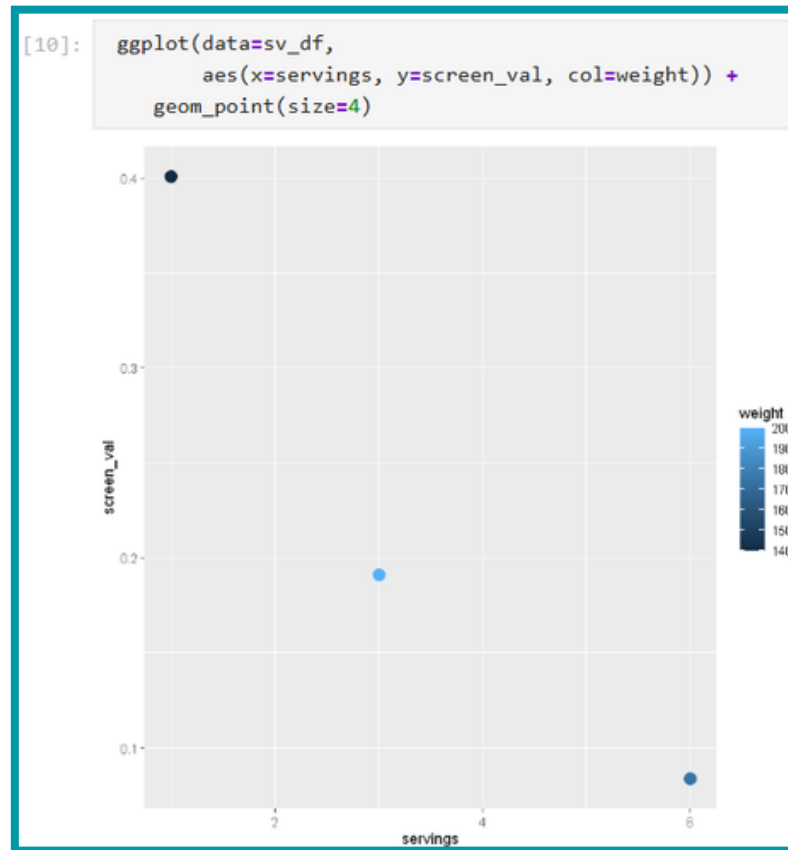


Add in the desired variables for the x and y-axis



Assign the type of plot you would like with “geom”

# Visualizing Screening Value vs. Servings Per Week with ggplot

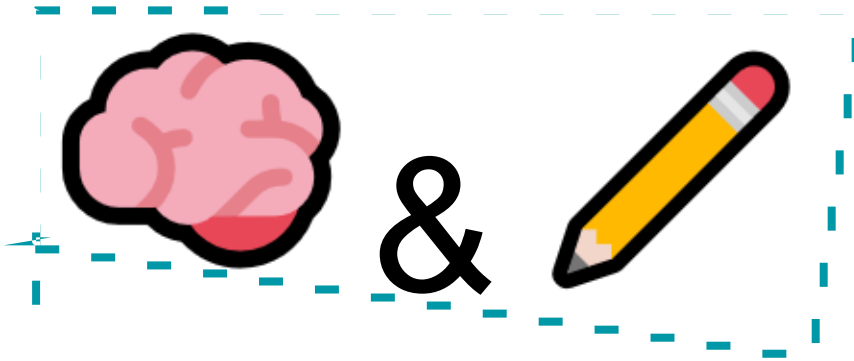


Increase point size

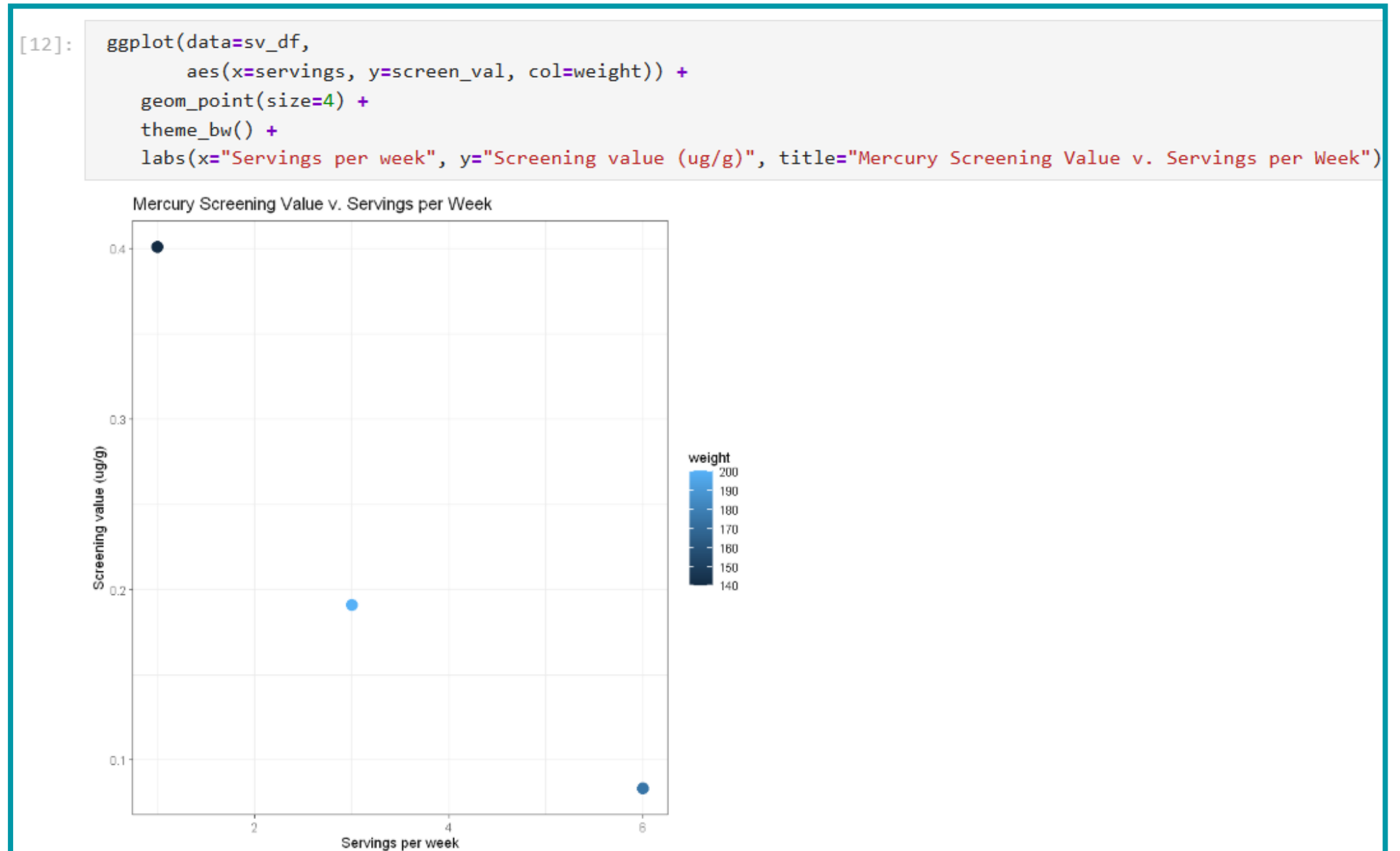
Change theme

Add title and  
descriptive axes  
labels

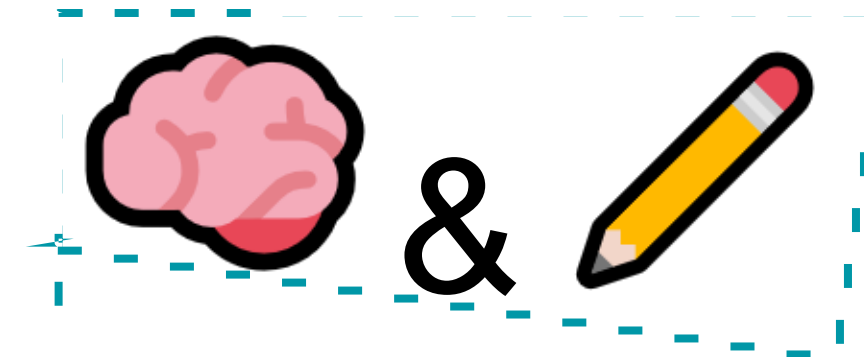
# Identify and describe relationships in a graph



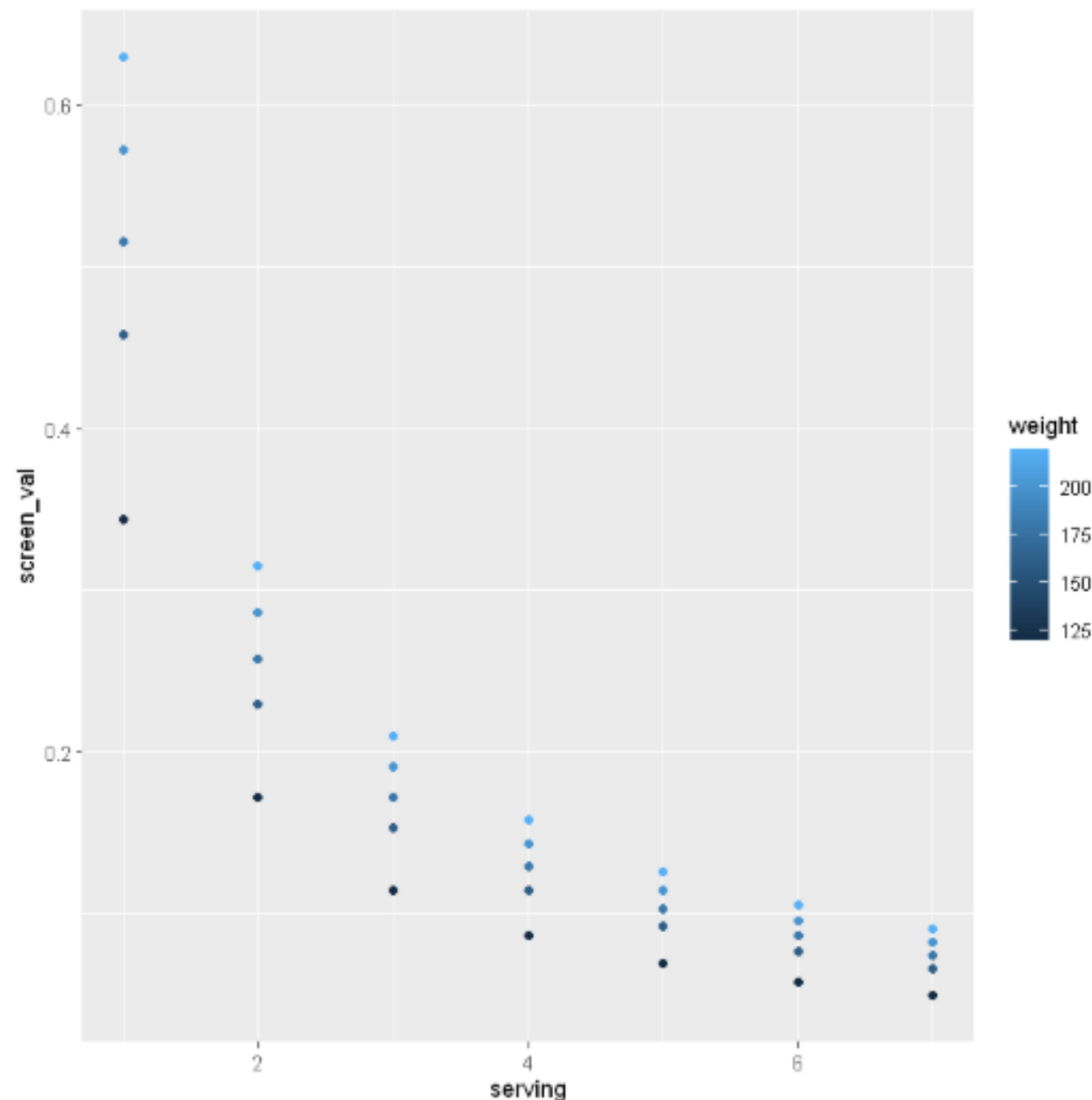
What is the relationship between screening value and the number of servings?



# Visualizing Screening Value vs. Servings Per Week with ggplot



```
[14]: weight = rep(c(120, 160, 180, 200, 220), 7)
 serving = rep(seq(1:7), 5)
 sv_df_long = data.frame(weight, serving)
 sv_df_long$screen_val = RfD*sv_df_long$weight*0.45/(110*sv_df_long$serving/7)
 ggplot(sv_df_long) + geom_point(aes(x=serving, y=screen_val, col=weight))
```



How would you describe this graph and the relationship between screening value, consumption rate, and weight? What happens to the distribution in screening values as the servings per week increase?

How should we consider the screening value for different ages? For example, what screening value would be safe for children, who weigh less than the average adult?

# Lesson 1.2: Load in and Explore 1998 Fish Data

## Goals:

- Load the 1998 fish data set
- View the entire data set in spreadsheet form
- View a truncated version of the data set
- Identify the number of rows and columns in the 1998 fish data set



# Loading and Exploring Data

1

```
[2]: # if running in google colab, uncomment and use the following line:
fish_1998 <- read.csv("https://raw.githubusercontent.com/rachtorr/IndigenousEnvDataSci.github.io/refs/heads/main/Mod1/fish_1998.csv")

read in CSV file
fish_1998 <- read.csv("fish_1998.csv", header = TRUE, sep = ',')
```

1. Run code to read in the csv (comma separated value) file

2. Double-click on the fish\_1998.csv file

3. The data will appear in a tab next to the python notebook (pynb)

2

The screenshot displays the Binder web interface. On the left, a file explorer shows the directory structure of the project. The file 'fish\_1998.csv' is highlighted with a purple circle. On the right, a Jupyter notebook titled 'Mod1\_pt2.ipynb' is open. A new tab for 'fish\_1998.csv' has been created and is also circled in purple. The notebook content includes a lesson title 'Lesson 1.2: Loading and exploring data' and a code cell with the same R code as shown in the first block.

Name	Modified
fish_1998.csv	last mo.
fish_2023.csv	last mo.
mod1_intro.md	last mo.
Mod1_pt1.ipynb	last mo.
Mod1_pt2.ipynb	next yr.
Mod1_pt3.ipynb	last mo.
Mod1_pt4.ipynb	last mo.
Mod1_pt5.ipynb	last mo.

**Lesson 1.2: Loading and exploring data**

These data are the length and mercury content of fish caught on the K'Avi reserva

- Each row is data from one fish.
- The first column "length" is the length of the fish in cm.
- The second column "mercury" is the concentration of mercury in the fish, in u

```
[1]: # if running in google colab, uncomment and use the following line:
fish_1998 <- read.csv("https://raw.githubusercontent.com/rachtorr/In
read in CSV file
```

3

# Loading and Exploring Data

```
fish_1998 <- read.csv("fish_1998.csv", header = TRUE, sep = ',')
```

Let's break down the different parts of what just happened:

1. We told R to read the data (a .csv file) using the command `read.csv()`
2. Within the `()`, we identify the data file from our working directory that we want to load into R
3. `header = TRUE` just tells R that the first row of data is the column names. The alternative is `header = FALSE`
4. `sep` is short for the word "separate", and it tells R that each column in the csv is separated by a comma `,`
3. `"fish"` is the name we are giving that file in R, and is how we will refer to this data going forward.
4. `<-` is like `=` in a math equation, it tells R that `"fish"` is the same as the data `"fish_1998.csv"`
5. This command turns a file on your computer (`fish_1998.csv`) into a dataframe (`fish_1998`)

# Previewing the Data

Run the code containing head and str commands.

`head()` shows the column names of the first 6 rows of data

`str()` shows the structure of the data, including column names, data type, and dimensions

300 rows of data

X = integer

length = numerical

mercury = numerical

```
[3]: head(fish_1998)

str(fish_1998)
```

**head(**

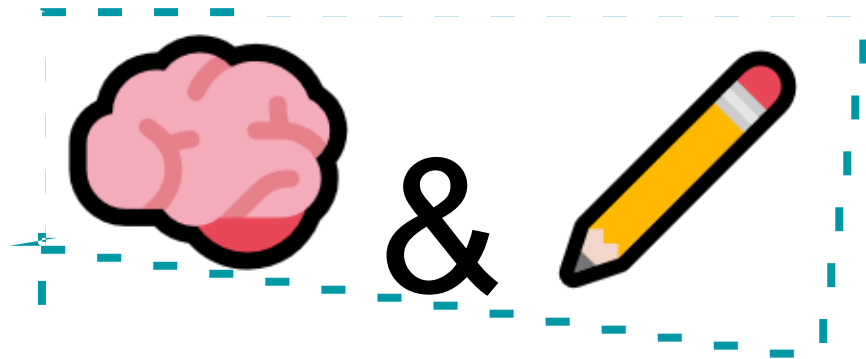
**str(**

A data.frame: 6 × 3

	X	length	mercury
	<int>	<dbl>	<dbl>
1	1	45.21643	0.4315457
2	2	44.34876	0.4969477
3	3	46.86563	0.4491865
4	4	41.67329	0.4137213
5	5	51.34514	0.7491135
6	6	56.66428	0.6807240

```
'data.frame': 300 obs. of 3 variables:
 $ X : int 1 2 3 4 5 6 7 8 9 10 ...
 $ length : num 45.2 44.3 46.9 41.7 51.3 ...
 $ mercury: num 0.432 0.497 0.449 0.414 0.749 ...
```

# Previewing the Data



What are the columns in the R dataframe called in fish\_1998?

What sorts of questions could the tribal fishery managers answer with this data?

```
[3]: head(fish_1998)

str(fish_1998)
```

A data.frame: 6 × 3

	X	length	mercury
	<int>	<dbl>	<dbl>
1	1	45.21643	0.4315457
2	2	44.34876	0.4969477
3	3	46.86563	0.4491865
4	4	41.67329	0.4137213
5	5	51.34514	0.7491135
6	6	56.66428	0.6807240

```
'data.frame': 300 obs. of 3 variables:
 $ X : int 1 2 3 4 5 6 7 8 9 10 ...
 $ length : num 45.2 44.3 46.9 41.7 51.3 ...
 $ mercury: num 0.432 0.497 0.449 0.414 0.749 ...
```

# Exploring Dimensions of the Data

Run the code section with the `dim()` command.

```
[6]: dim(fish_1998)
300 . 3
```

This command shows the dimensions of the data set

Dimension = the number of rows and number of columns.

# Terms to Learn for Running Code

- **Variable:** A storage for data that can be referenced later.
- **Function:** A reusable piece of code that performs a specific task.
- **Library/Package:** A collection of functions and tools that extend R's capabilities.
- **Data Frame:** A table-like structure used to store and manipulate data.
- **CSV (Comma-Separated Values):** A common file format for data storage.
- **GitHub Repository:** A cloud-based storage for sharing and managing code.
- **Runtime:** The environment where code executes.
- **Install Packages:** The process of adding additional functionalities in R.
- **Execute/Run Code:** The action of processing code to get results.
- **Kernel:** The computational engine that runs your notebook.

# Key Terms for Beginners Using Tidyverse & Lubridate

- **Tidyverse:** A collection of R packages for data manipulation, visualization, and analysis.
- **ggplot2:** A package for creating static and interactive visualizations.
- **dplyr:** A package for data manipulation using functions like `filter()`, `mutate()`, `select()`, and `summarize()`.
- **tidyr:** A package for cleaning and reshaping data using functions like `spread()`, `gather()`, `separate()`, and `unite()`.
- **readr:** A package for reading in CSV and other text-based files.
- **lubridate:** A package for handling and manipulating date-time data.
- **ymd()**, **mdy()**, **dmy()**: Functions in lubridate for parsing dates.
- **mutate()**: A dplyr function for adding new columns or modifying existing ones.
- **group\_by()**: A dplyr function for grouping data before applying summarizing operations.
- **summarize()**: A dplyr function for calculating summary statistics like mean, median, or count.
- **pipe (%>%)**: A Tidyverse operator used to pass data from one function to another.